



សាលាបង្វែរស្ទីវអាឡធី
បង្រៀនជំនាញ បង្កើតអាជីពសម្រាប់ការងារនិងមុខរបរ

វគ្គជំនាញអភិវឌ្ឍន៍គេហទំព័រ

E-Commerce Website

ជួយឱ្យអ្នកសិក្សាបានបទពិសោធន៍ពីការអនុវត្តបង្កើត E-Commerce Website បែបស្តង់ដារ



Vue.js



Tailwind css



Firebase

2024 - 2025



មាតិកា

មាតិកា	I
ជំពូកទី១ VUE JS	1
មេរៀនទី១ និយមន័យ និងការដំឡើង	1
១. និយមន័យ VUE និង VUE/CLI	1
២. ការដំឡើង NODE និង VUE/CLI	1
២.១ របៀបដំឡើង NODE	1
២.២ របៀបដំឡើង VUE/CLI	8
មេរៀនទី២ មូលដ្ឋានគ្រឹះ VUE.JS	9
១. ការបង្កើត VUE Project	9
២. Template និង Data	12
៣. Methods និង Click Events	13
៤. Conditional Rendering	14
៥. Mouse Events	15
៦. List (v-for)	16
៧. Attribute Binding	17
៨. Static និង Dynamic classes	19
៩. Computed Property និង V-Model	20
១០. The Different Between Computed and Methods Property	23
មេរៀនទី៣ កម្រិតថ្នាក់មធ្យម INTERMEDIATE	25
១. Template Refs	25
២. Component និង Multi Components	26
៣. Dynamic Components	30
៤. Component Styles និង Global Styles	31
៥. Passing Data with Props	33
៦. Emitting Events	34
៧. Event Modifiers	35
មេរៀនទី៤ ទម្រង់បញ្ចូលទិន្នន័យ FORM	38
១. ការប្រើប្រាស់និងការណែនាំ	38
២. Select Fields, Checkbox, និង Keyboard Events	39
មេរៀនទី៥ VUE ROUTER	43

១. ហេតុអ្វីប្រើ Vue Router និងការបញ្ចូលវាទៅក្នុង Project.....	43
២. Dynamic និង Static Router Link.....	43
៣. Route Parameters.....	45
ជំពូកទី២ TAILWINDCSS	48
មេរៀនទី១ ការនិយមន័យ និងដំឡើង.....	48
១. តើអ្វីទៅជា Tailwindcss?.....	48
២. ការដំឡើង	48
៣. Configuration.....	48
មេរៀនទី២ CORE CONCEPTS	51
១. ហ្វុនអក្សរ Typography.....	51
២. ពណ៌ Colors	53
៣. Background.....	54
៤. ទំហំទទឹង Width	55
៥. ទំហំកម្ពស់ Height.....	55
៦. Padding	55
៧. Margin.....	56
៨. Spacing	56
៩. Border	56
១០. ក្រឡាចត្រង្គ Grid	57
១១. Flex Box	59
១២. Breakpoint.....	61
ជំពូកទី៣ FIREBASE	62
មេរៀនទី១ និយមន័យ និងការដំឡើង.....	62
១. និយមន័យ	62
២. ការដំឡើង	62
មេរៀនទី២ CRUD Operations	71
១. Create Operation.....	71
២. Read Operation.....	71
៣. Update Operation	72
៤. Delete Operation	72



ជំពូកទី១

VUE JS

មេរៀនទី១

និយមន័យ និងការដំឡើង

១. និយមន័យ VUE និង VUE/CLI

VUE គឺជា Progressive Framework សំរាប់អ្នកប្រើប្រាស់ភាសា JavaScript ដើម្បីបង្កើត Web Interfaces និង Single Page Application ។

VUE/CLI គឺជាការដំឡើងតាមរយៈ Node Package Management ជាលក្ខណៈ Globally ដែលអនុញ្ញាតឲ្យយើងប្រើប្រាស់ពាក្យបញ្ជារបស់ VUE នៅក្នុង Terminal ឬ Command Prompt (CMD) បាន។ វាមានសមត្ថភាពបង្កើតសំបក Project ដែលមាន Structure រួចជាស្រេចតាមរយៈពាក្យបញ្ជា `vue create` ។

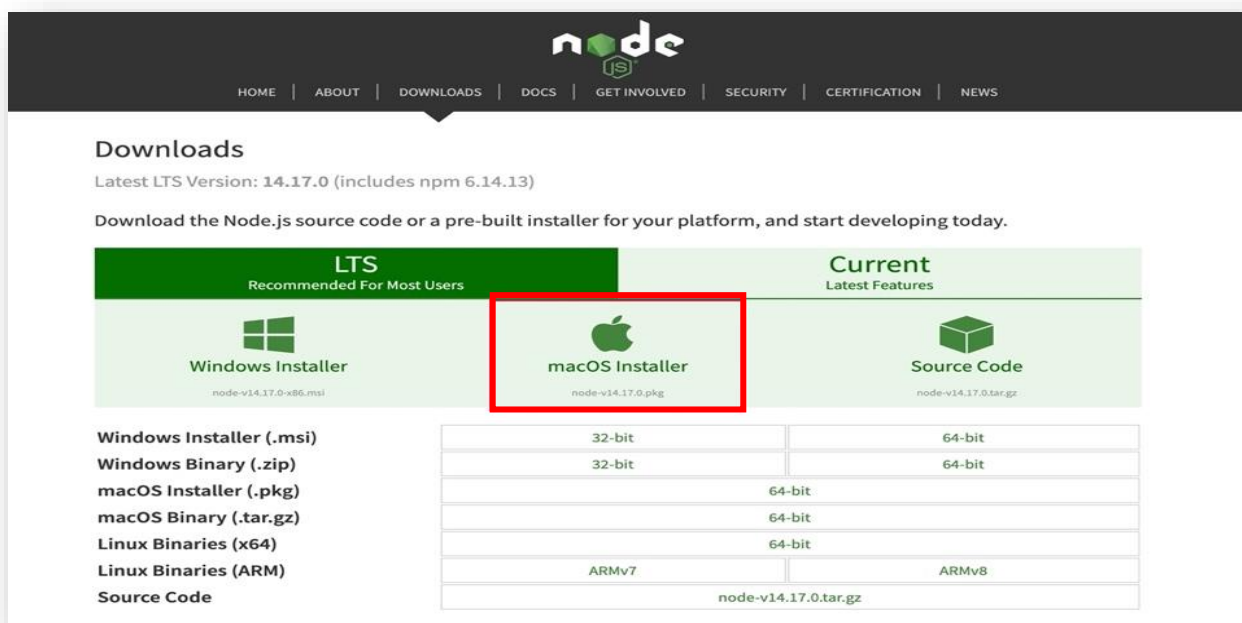
២. ការដំឡើង NODE និង VUE/CLI

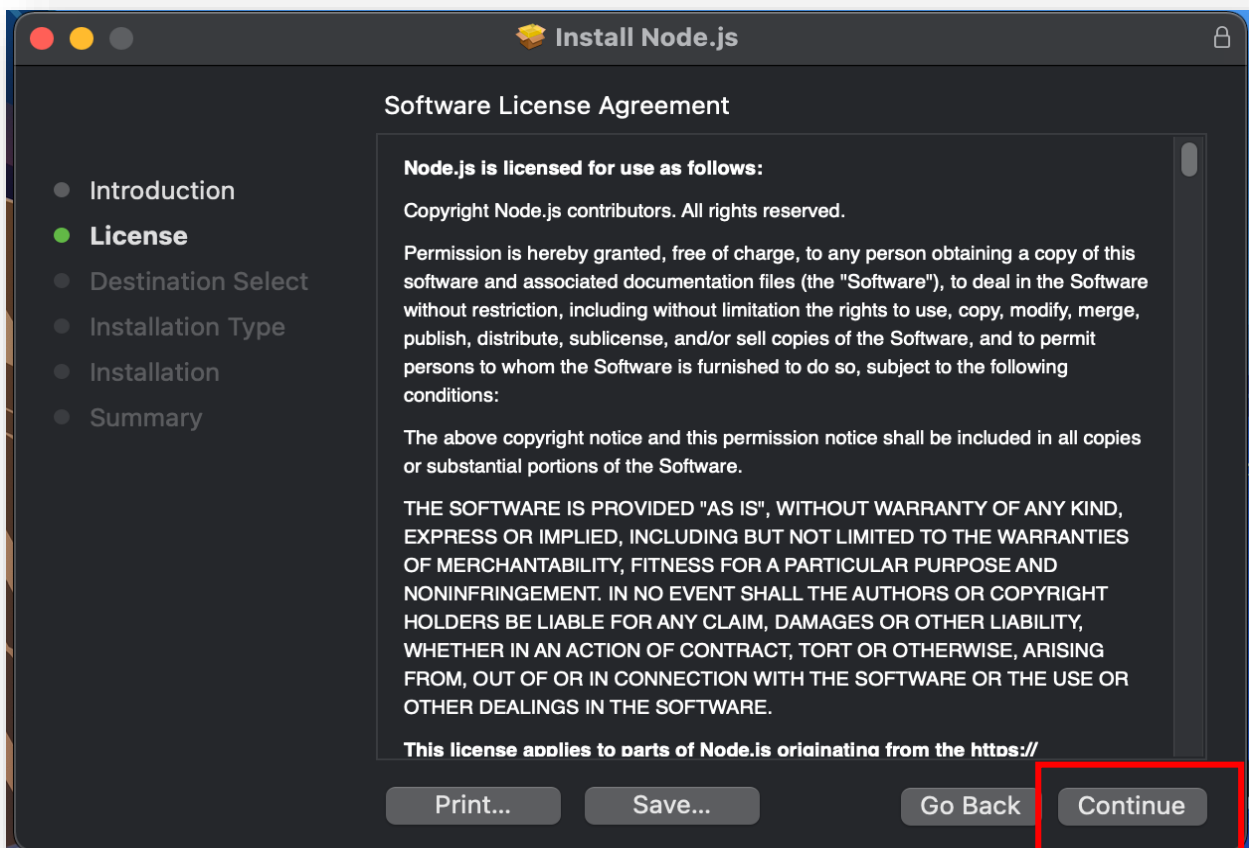
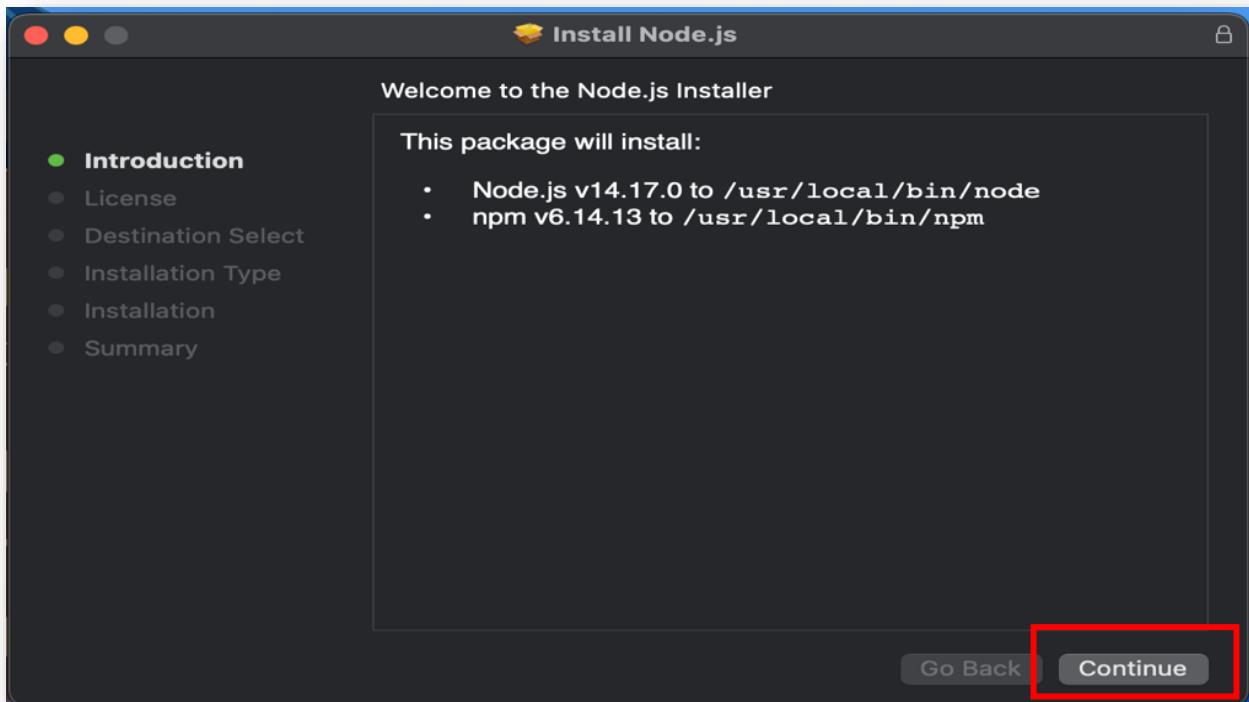
២.១ របៀបដំឡើង NODE

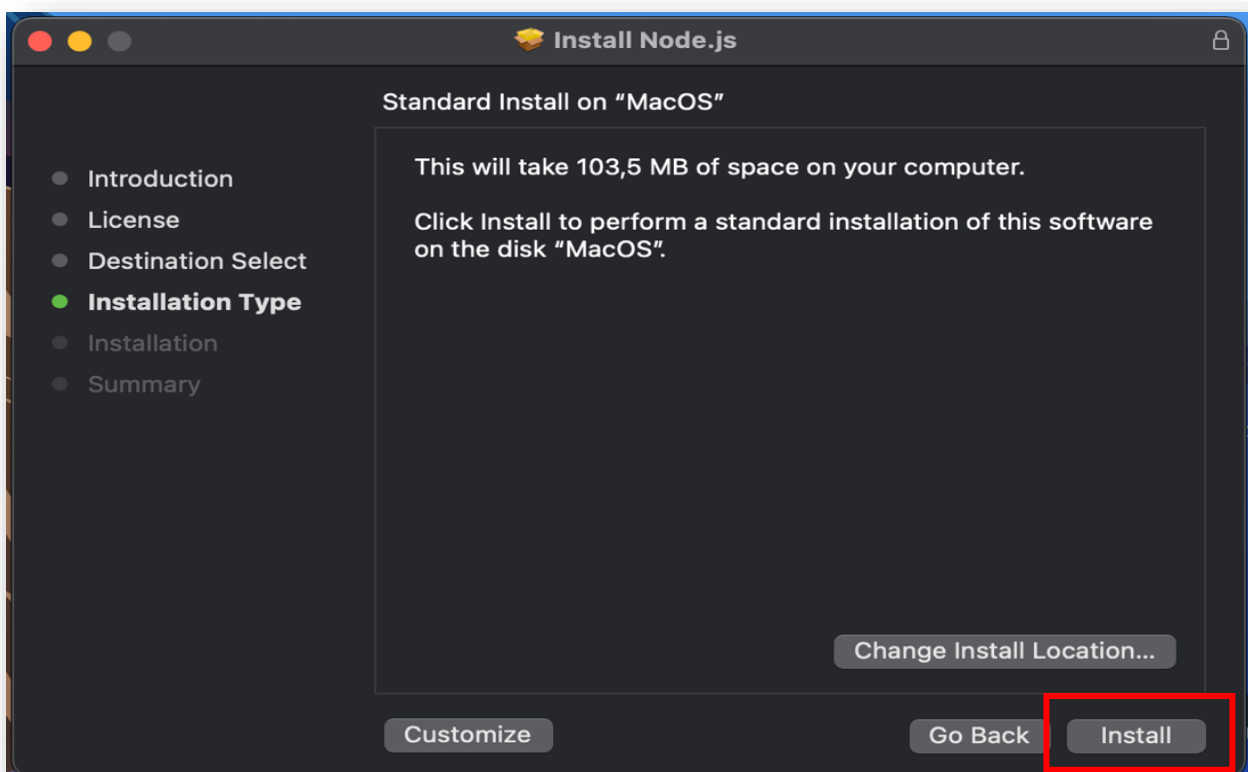
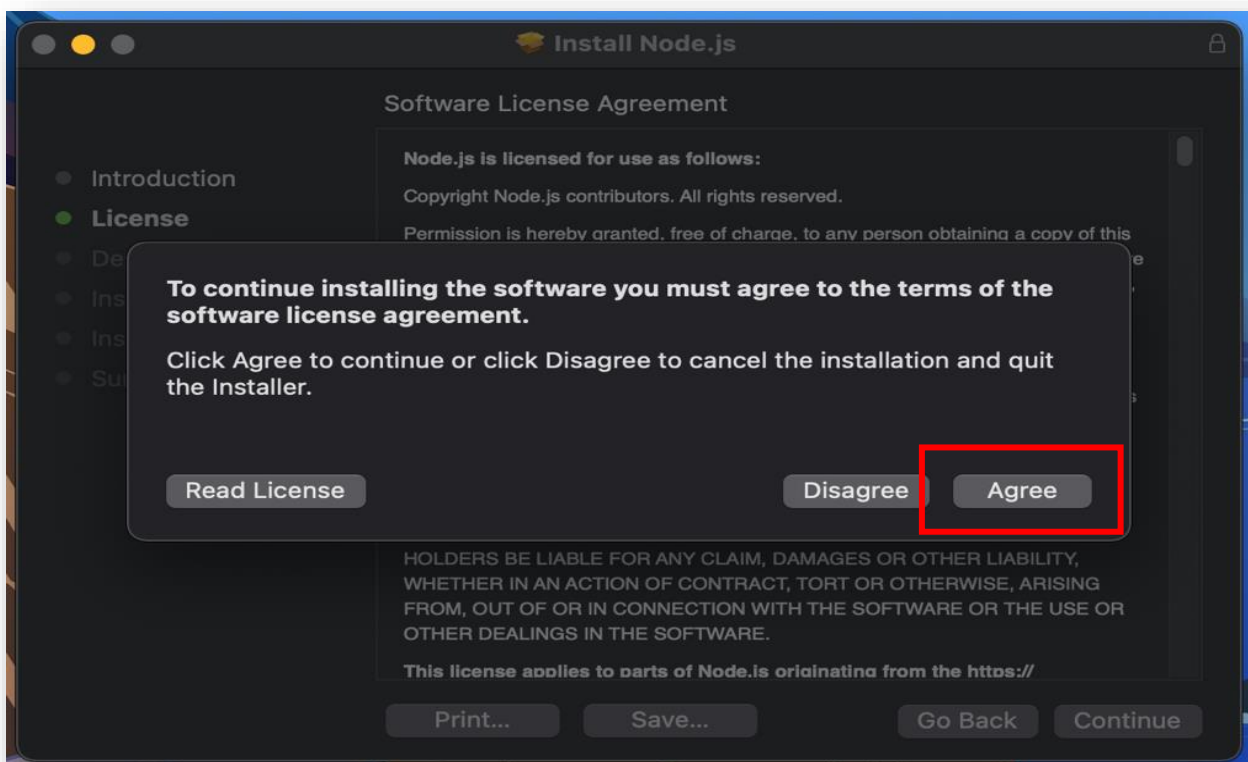
+ សំរាប់អ្នកប្រើប្រាស់ macOS

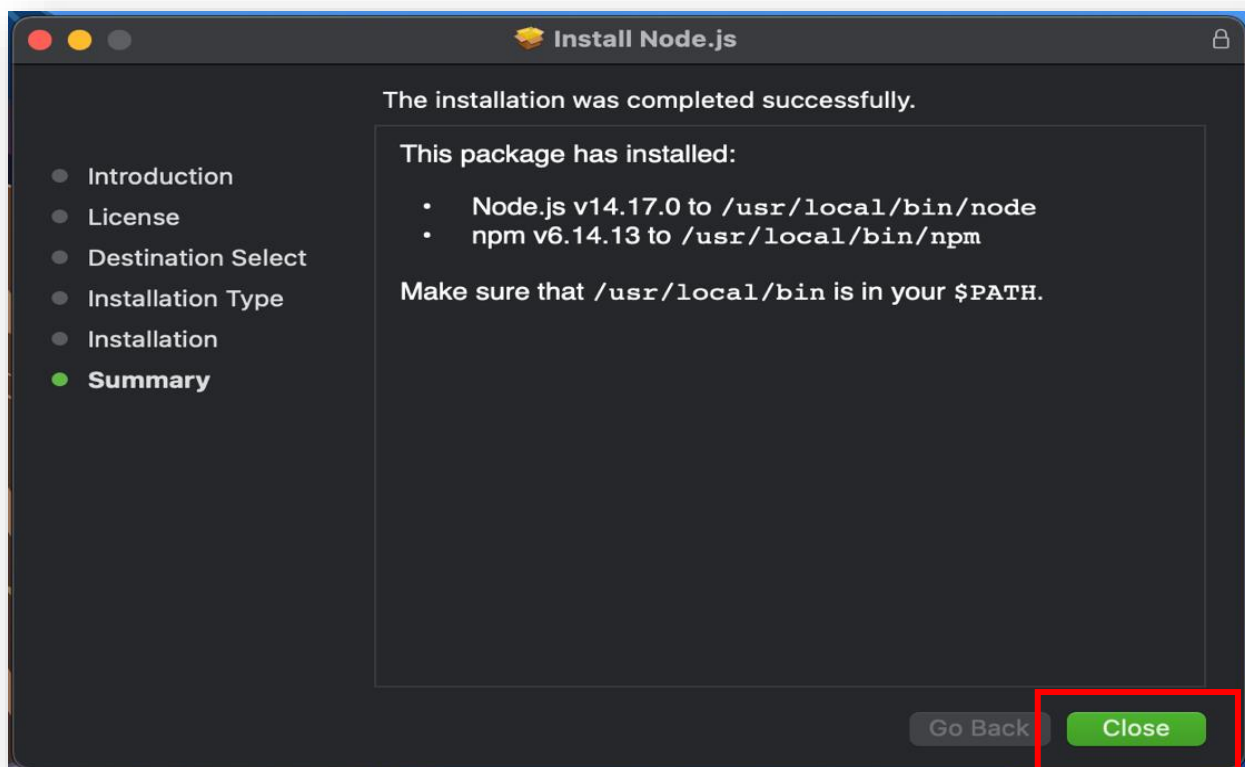
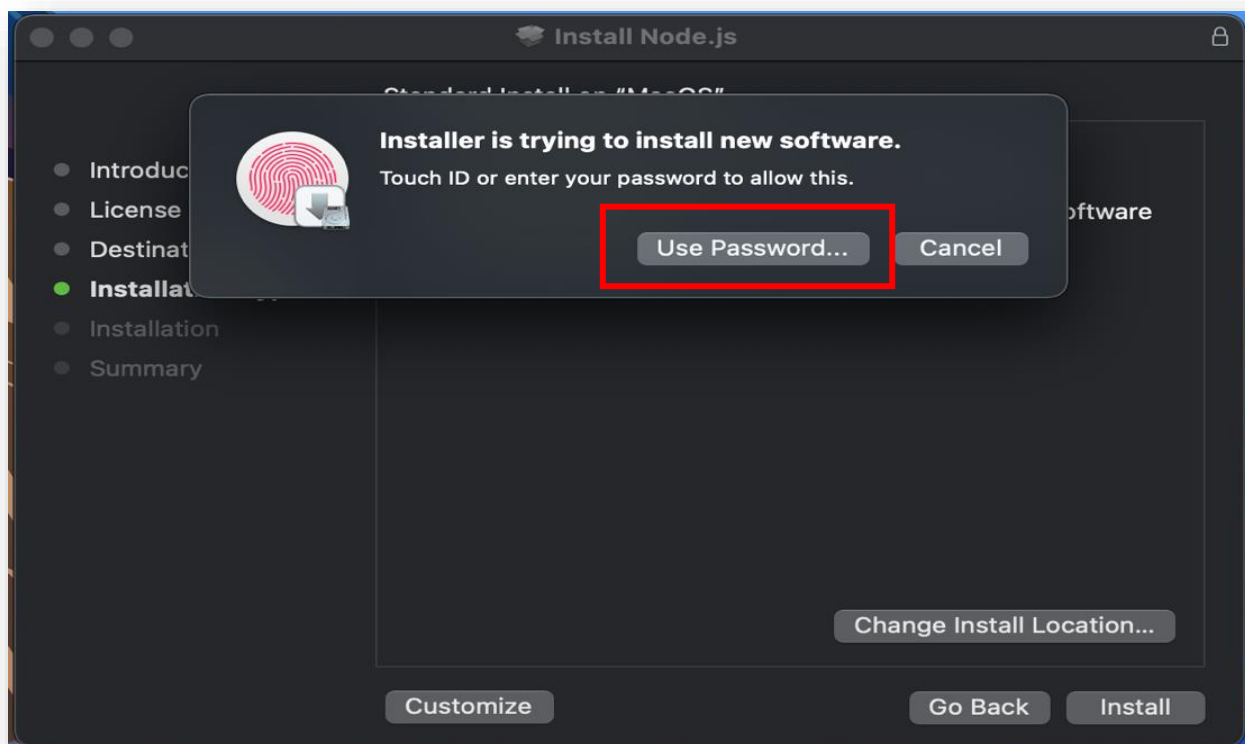
ចូរទៅកាន់ browser រួចវាយបញ្ចូល <https://nodejs.org/en/download/>

រួចអនុវត្តតាមរូបខាងក្រោមនេះ៖





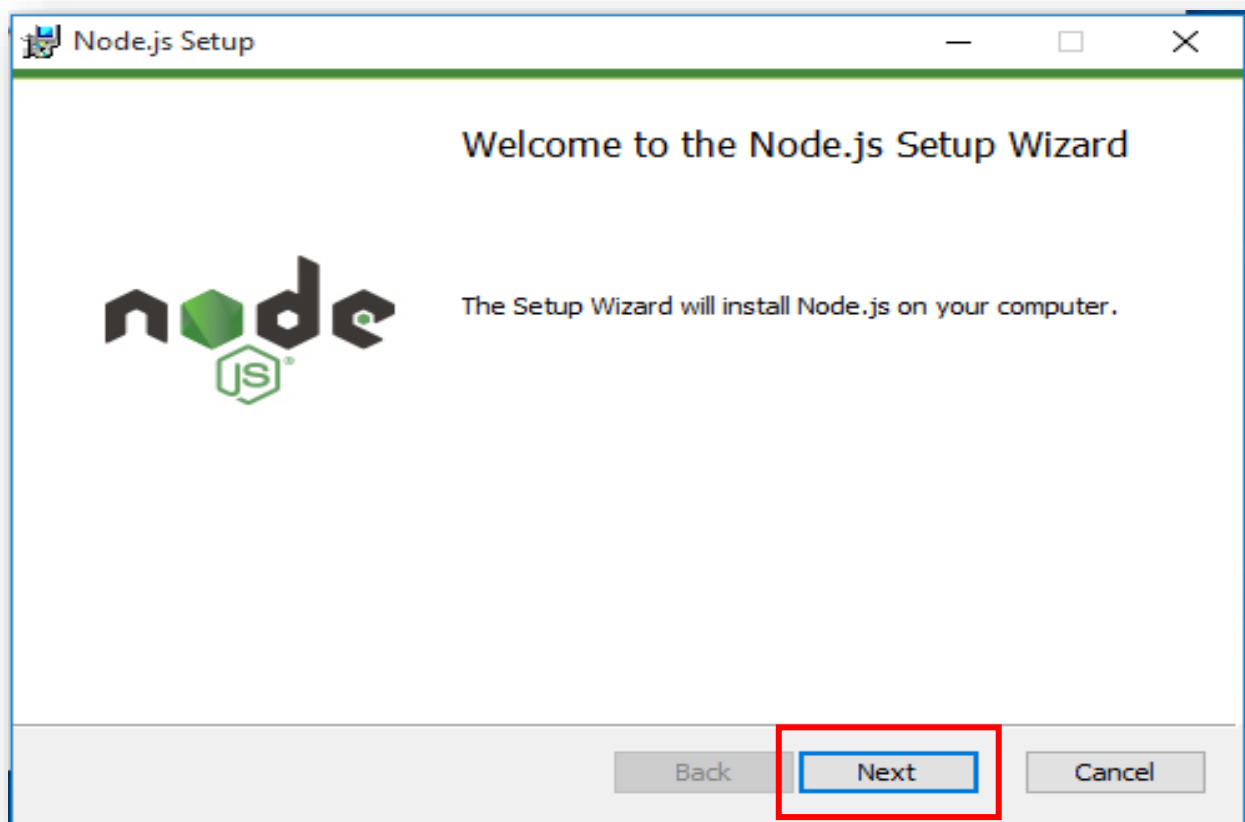
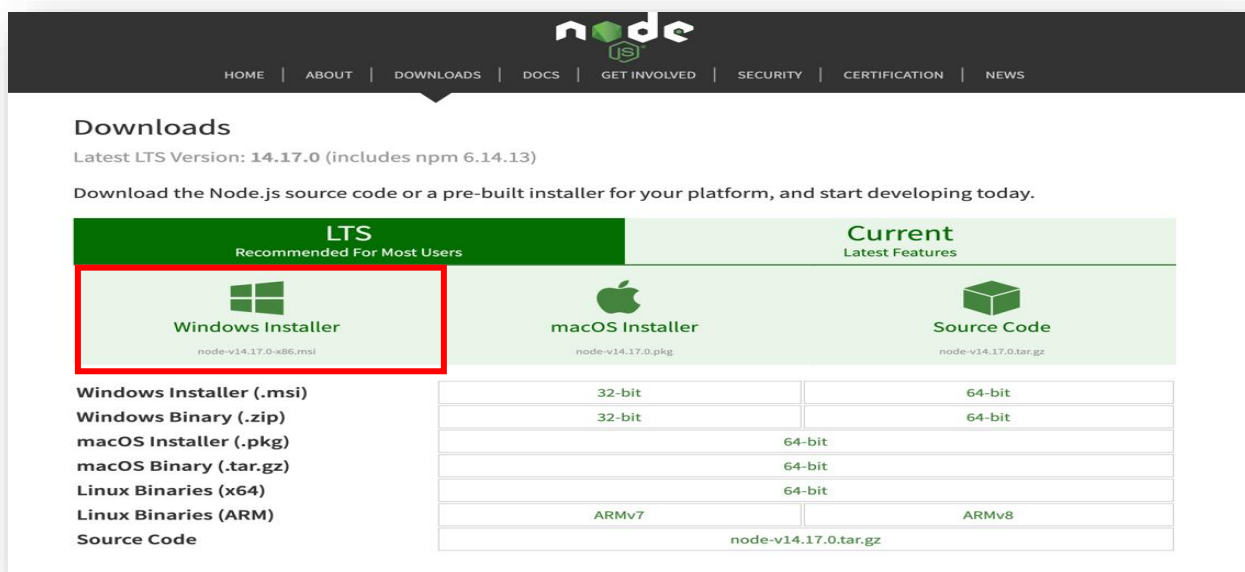


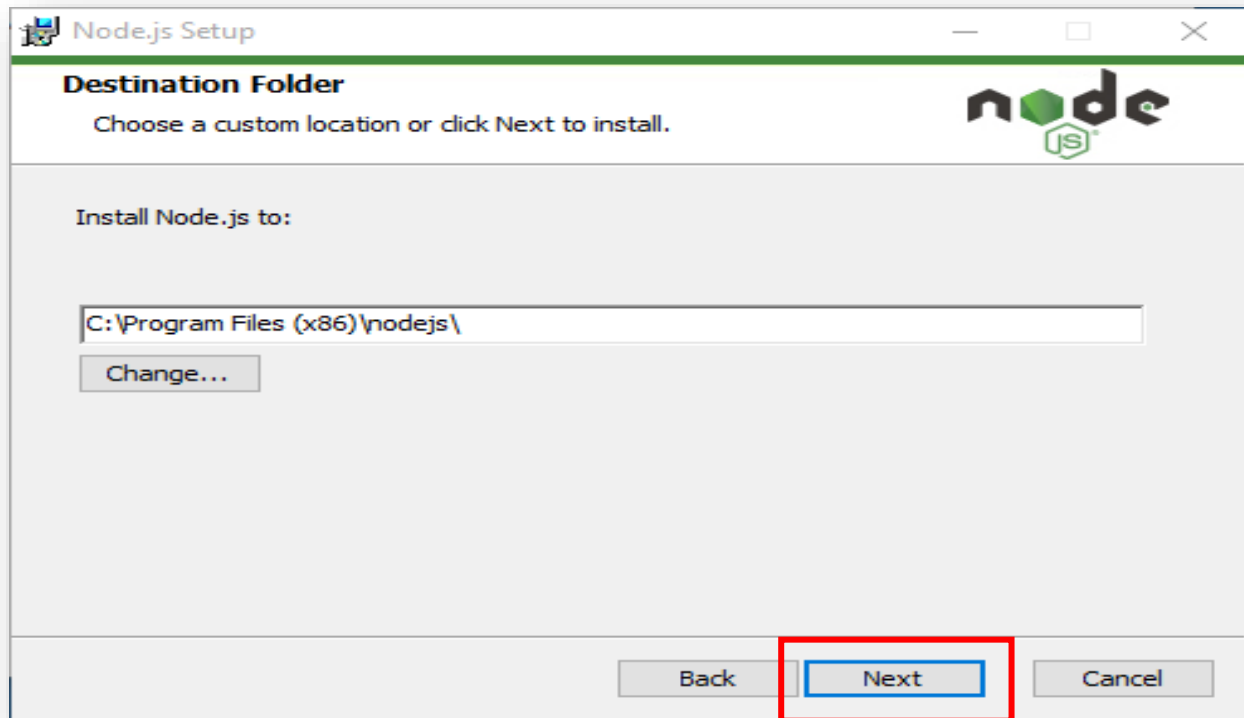
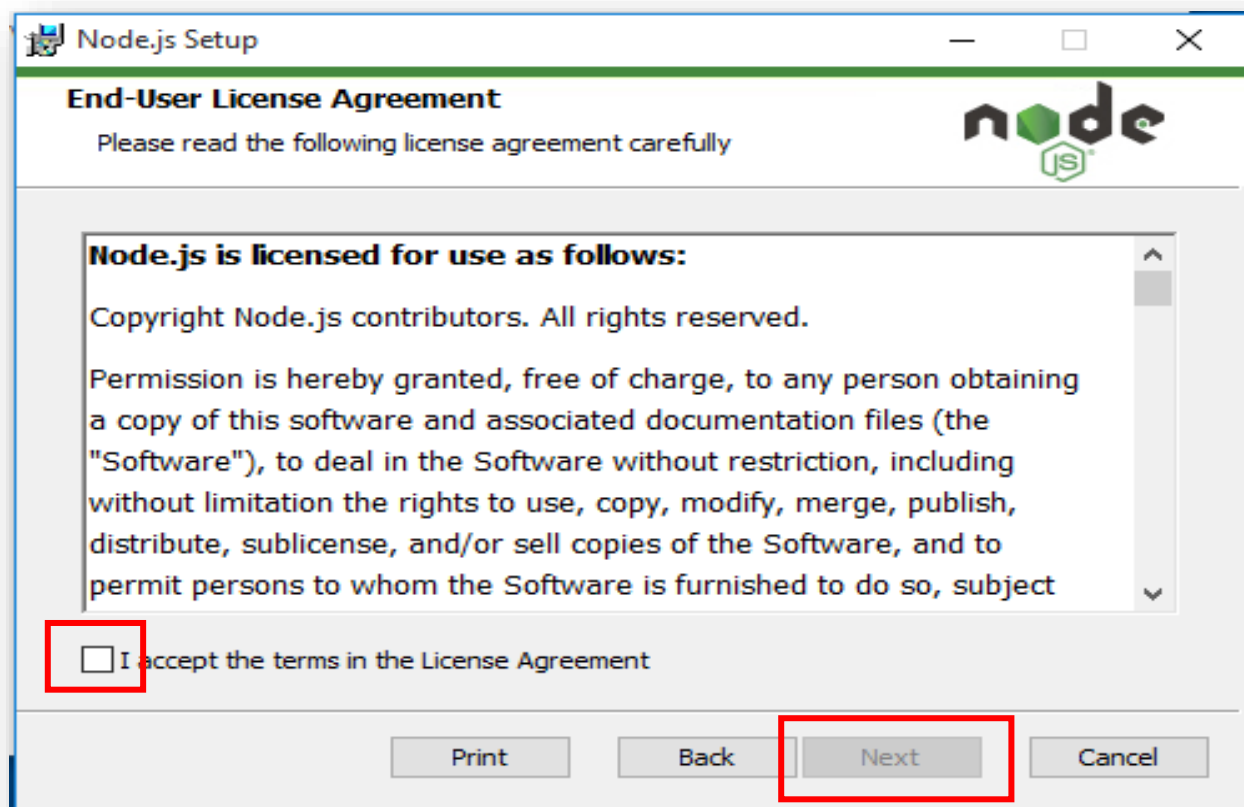


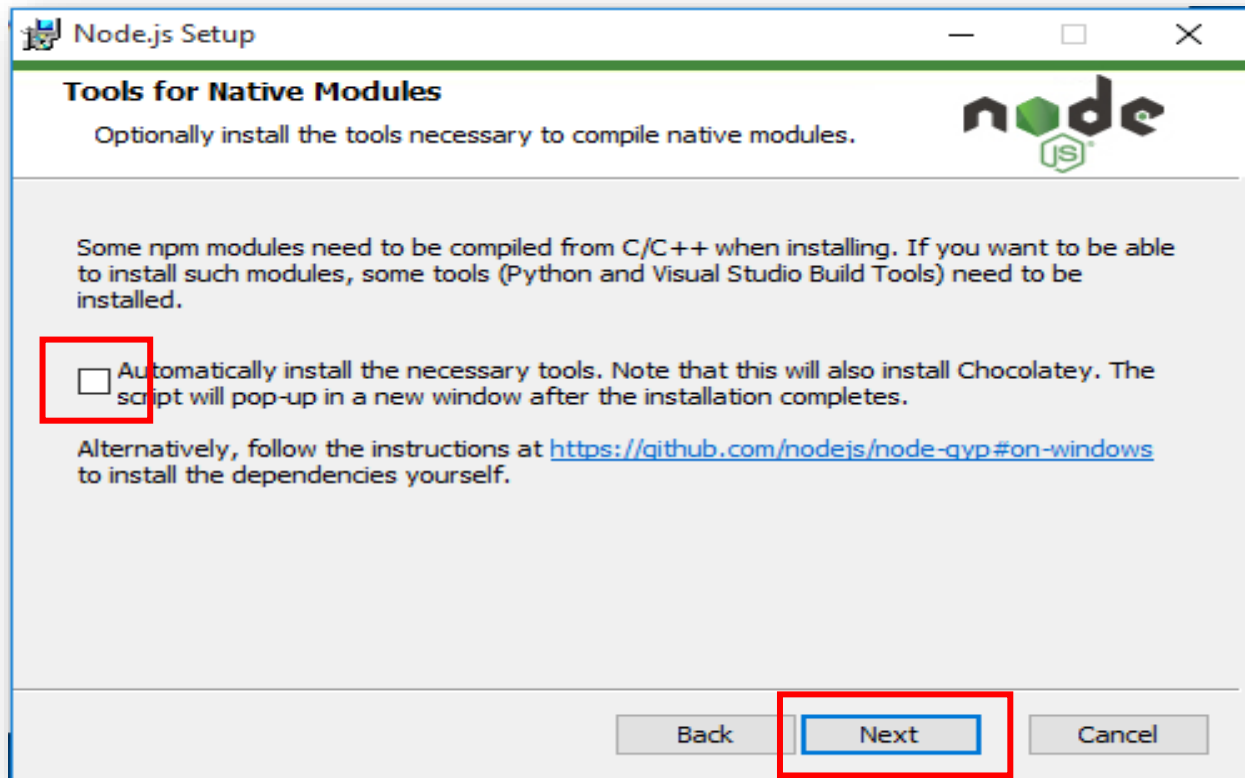
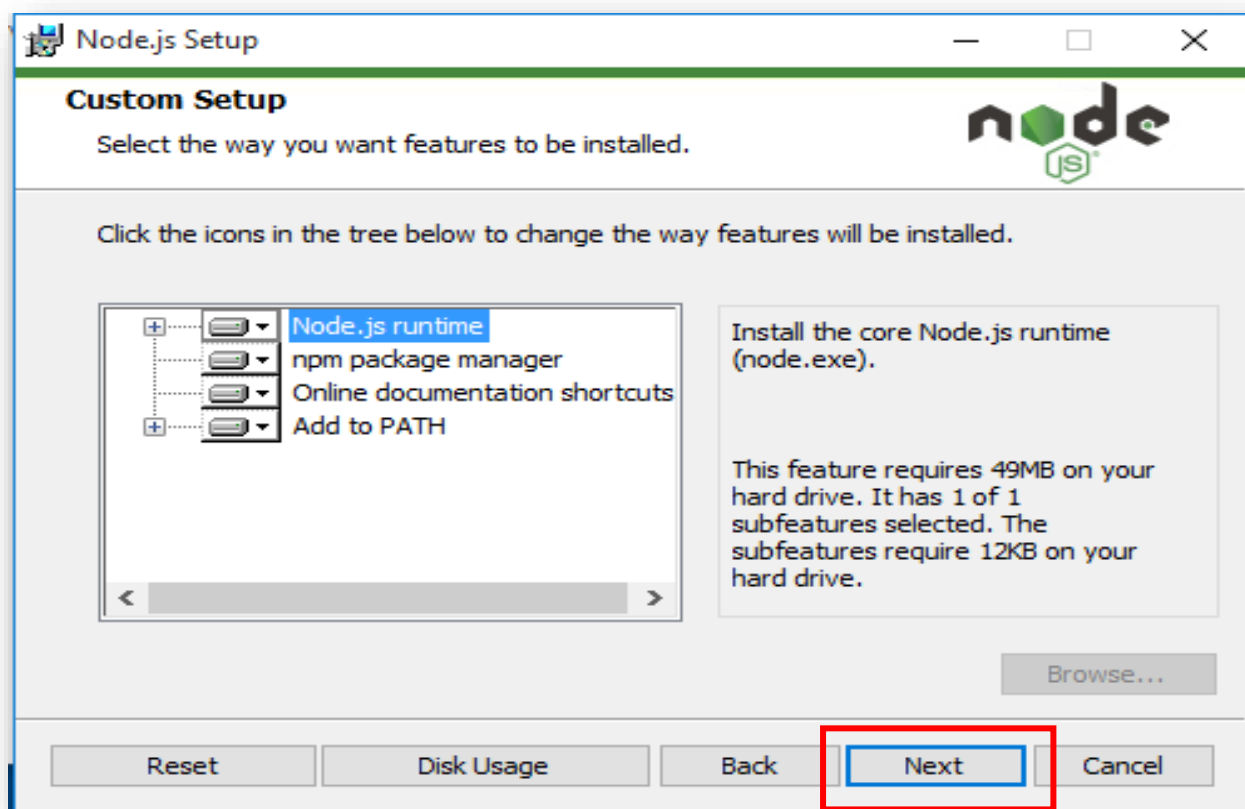
+ សំរាប់អ្នកប្រើប្រាស់ Window OS

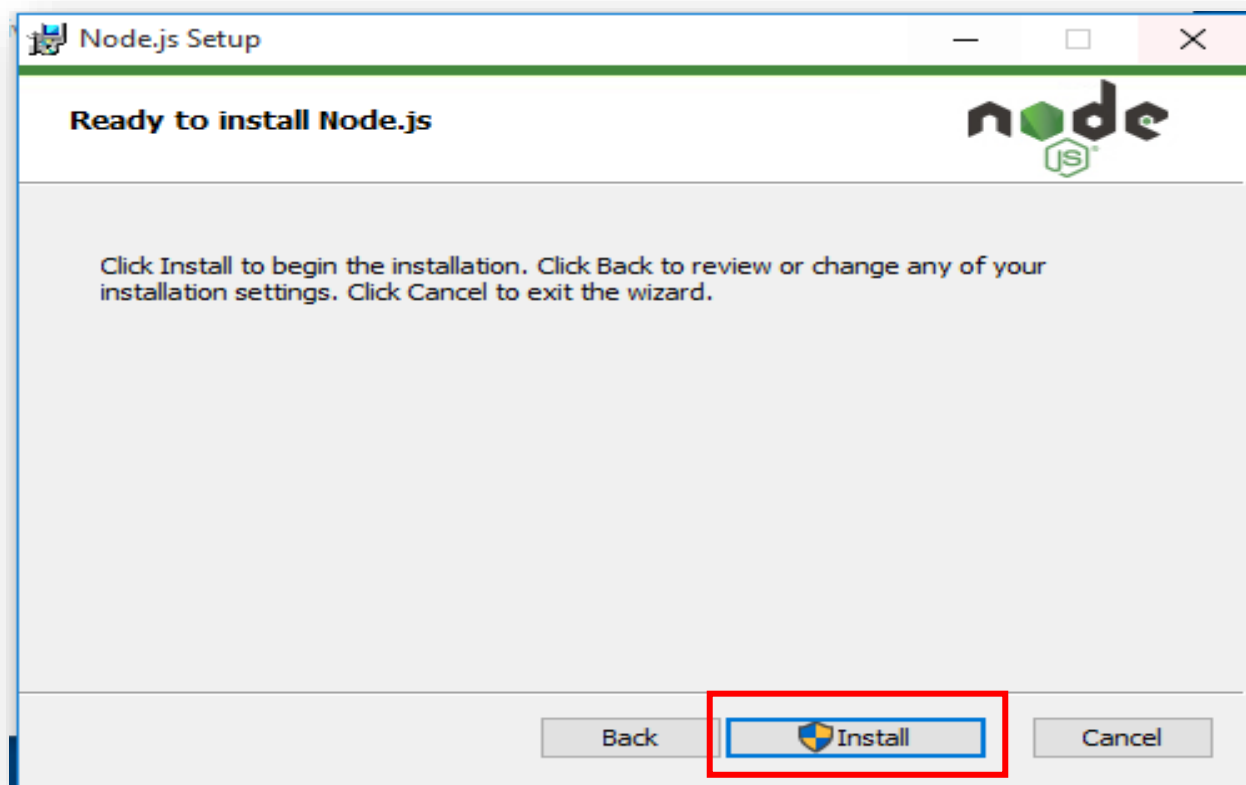
ចូរទៅកាន់ browser រួចវាយបញ្ចូល <https://nodejs.org/en/download/>

រួចអនុវត្តតាមរូបខាងក្រោមនេះ៖









២.២ របៀបដំឡើង VUE/CLI

បន្ទាប់ពីដំឡើង Node ហើយយើងបាន npm មកជាមួយដែរ។ ដើម្បីដំឡើង VUE/CLI យើងប្រើពាក្យបញ្ជា `npm install -g @vue/cli` នៅក្នុង Terminal (macOS) ឬ Command Prompt (windowOS) ។

មេរៀនទី២

មូលដ្ឋានគ្រឹះ VUE.JS

១. ការបង្កើត VUE Project

មុនដំបូងចូលទៅកាន់ Command Prompt ឬ Terminal បន្ទាប់មកវាយបញ្ចូលពាក្យបញ្ជា `vue create project_name` ។ ខាងក្រោមនេះគឺជាគំរូ៖

```

thearith@Phengs-MacBook-Pro ~/Desktop$ vue create ecommerce-project

Vue CLI v4.5.12

New version available 4.5.12 → 4.5.13

? Please pick a preset: (Use arrow keys)
> no ([Vue 3] babel, vuex)
  Default ([Vue 2] babel, eslint)
  Default (Vue 3 Preview) ([Vue 3] babel, eslint)
  Manually select features
  
```

```

Vue CLI v4.5.12

New version available 4.5.12 → 4.5.13

? Please pick a preset: Manually select features
? Check the features needed for your project:
  ● Choose Vue version
  ● Babel
  ○ TypeScript
  ○ Progressive Web App (PWA) Support
  ● Router
  ○ Vuex
  ○ CSS Pre-processors
  ○ Linter / Formatter
  ○ Unit Testing
  ○ E2E Testing
  
```




```
Desktop — vue create ecommerce-project — node ~/.nvm/versions/node/v15.2.1/bin/vue create ecommerce-project — 128x31
vue
Vue CLI v4.5.12

New version available 4.5.12 → 4.5.13

? Please pick a preset: Manually select features
? Check the features needed for your project: Choose Vue version, Babel, Router
? Choose a version of Vue.js that you want to start the project with
  4.x
> 3.x (Preview)
```

```
Desktop — vue create ecommerce-project — node ~/.nvm/versions/node/v15.2.1/bin/vue create ecommerce-project — 128x31
vue
Vue CLI v4.5.12

New version available 4.5.12 → 4.5.13

? Please pick a preset: Manually select features
? Check the features needed for your project: Choose Vue version, Babel, Router
? Choose a version of Vue.js that you want to start the project with 3.x (Preview)
? Use history mode for router? (Requires proper server setup for index fallback in production) Yes
? Where do you prefer placing config for Babel, ESLint, etc.? (Use arrow keys)
> In dedicated config files
  In package.json
```

```

Vue CLI v4.5.12

New version available 4.5.12 -> 4.5.13

? Please pick a preset: Manually select features
? Check the features needed for your project: Choose Vue version, Babel, Router
? Choose a version of Vue.js that you want to start the project with 3.x (Preview)
? Use history mode for router? (Requires proper server setup for index fallback in production) Yes
? Where do you prefer placing config for Babel, ESLint, etc.? In dedicated config files
? Save this as a preset for future projects? (y/N) n

```

```

> node ./postinstall.js

added 1214 packages from 650 contributors and audited 1214 packages in 55.177s

68 packages are looking for funding
  run `npm fund` for details

found 108 moderate severity vulnerabilities
  run `npm audit fix` to fix them, or `npm audit` for details
  Invoking generators...
  Installing additional dependencies...

added 30 packages from 50 contributors and audited 1244 packages in 13.853s

69 packages are looking for funding
  run `npm fund` for details

found 108 moderate severity vulnerabilities
  run `npm audit fix` to fix them, or `npm audit` for details
  Running completion hooks...

Generating README.md...

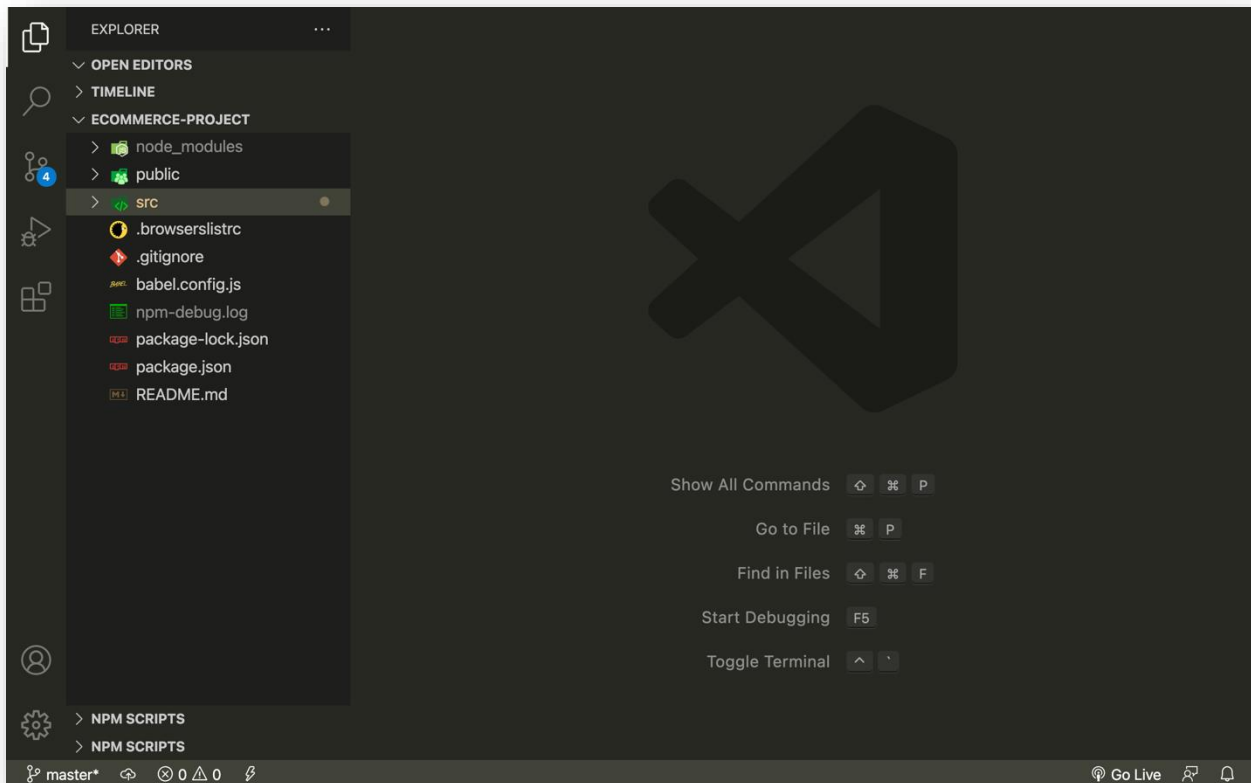
Successfully created project ecommerce-project.
Get started with the following commands:

$ cd ecommerce-project
$ npm run serve

thearith@Phengs-MacBook-Pro ~/Desktop $ cd ecommerce-project
thearith@Phengs-MacBook-Pro ~/Desktop/ecommerce-project $ npm run serve

```

ខាងក្រោមនេះគឺជាសំបក (scaffold) Project ដែលយើងបានបង្កើត៖



២. Template និង Data

Template នៅក្នុង Vue គឺជាការសរសេរ HTML ឬ ជាការបញ្ចូលគ្នារវាង HTML និង CSS នៅក្នុង template tag, ចំណែក Data គឺជា Dynamic ទិន្នន័យដែលយើងយកមកប្រើប្រាស់ ឬ បង្ហាញនៅលើ Browser តាមរយៈ template ។

ឧទាហរណ៍៖

```
<template>
  <h1>Hello Vue.js</h1>
  <h3>Author: {{ author }}</h3>
  <h3>Title: {{ title }}</h3>
</template>

<script>
import { ref } from "@vue/reactivity";
export default {
  setup() {
    const author = ref("មកព");
    const title = ref("Web Developer");

    return { author, title };
  },
};
</script>

<style>
```

```
div {
  padding: 20px;
}
h3 {
  color: blue;
  font-size: 24px;
}
</style>
```

៣. Methods និង Click Events

Methods គឺជាបណ្តុំនៃកូដដែលប្រើក្នុងគោលបំណងដាក់លាក់ណាមួយនិងត្រូវហៅមកប្រើនៅពេលមាន Event ណាមួយកើតឡើងហើយយើងសរសេរវានៅក្នុង methods property របស់ Vue ។

Click Events គឺជា Event ដែលនឹងកើតឡើងនៅពេលយើងចុចទៅលើធាតុ (element) ណាមួយនៅលើ Browser ។

ឧទាហរណ៍៖

```
<template>
  <h1>Hello Vue.js</h1>
  <h3>Author: {{ author }}</h3>
  <h3>Title: {{ title }}</h3>
  <button v-on:click="handleChangeTitle">Change Title</button>
</template>

<script>
import { ref } from "@vue/reactivity";
export default {
  name: "App",
  setup() {
    const author = ref("មករា");
    const title = ref("Web Developer");

    const handleChangeTitle = () => {
      title.value = "Web Designer";
    };
    return { author, title, handleChangeTitle };
  },
};
</script>

<style>
div {
  padding: 20px;
}
h3 {
  color: blue;
  font-size: 24px;
}
</style>
```



៤. Conditional Rendering

Conditional Rendering គឺជាលក្ខខណ្ឌសំរាប់បង្ហាញឬមិនបង្ហាញ User Interface (UI) អាស្រ័យលើលក្ខខណ្ឌពិតឬមិនពិត។

ឧទាហរណ៍៖

```
<template>
  <h1>Hello Vue.js</h1>
  <h3>Author: {{ author }}</h3>
  <h3>Title: {{ title }}</h3>
  <button v-on:click="handleToggleAuthor">Toggle Author</button>
</template>

<script>
import { ref } from "@vue/reactivity";
export default {
  name: "App",
  setup() {
    const author = ref("មកក");
    const title = ref("Web Developer");
    const toggle = ref(false);

    const handleToggleAuthor = () => {
      toggle.value = !toggle.value;
    };
    return { author, title, toggle, handleToggleAuthor };
  },
};
</script>

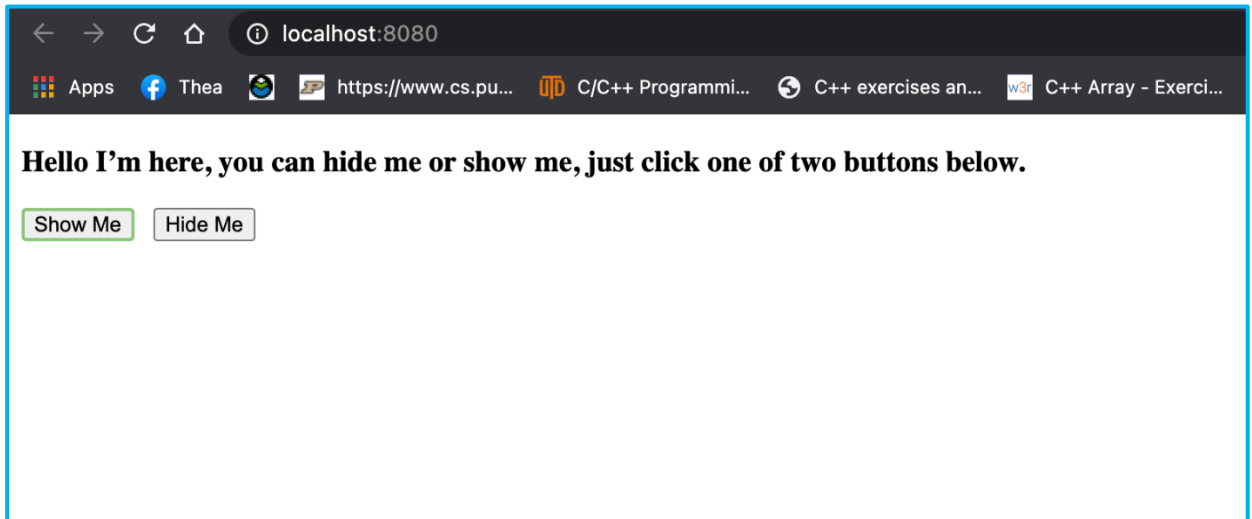
<style>
div {
  padding: 20px;
}
h3 {
  color: blue;
  font-size: 24px;
}
</style>
```

លំហាត់

ចូរបង្កើត button ពីរ, button ទី១ មានឈ្មោះថា Show Me, button ទី២ មានឈ្មោះថា Hide Me, ដែលនៅខាងលើ button ទាំងពីរមានសរសេរអក្សរមួយឃ្លាថា “Hello I’m here, you can hide me or show me, just click one of the buttons below.” ។

* បញ្ជាក់ button ទី១ ដែលមានឈ្មោះថា Show Me មានមុខងារសំរាប់បង្ហាញឃ្លាខាងលើចំនែក button ទី២ដែលមានឈ្មោះថា Hide Me មានមុខងារសំរាប់បិទបាំង(លាក់)ឃ្លាខាងលើ ។

ខាងក្រោមនេះគឺជាគំរូលទ្ធផល៖



៥. Mouse Events

Mouse Events កើតឡើងនៅពេលយើងប្រើប្រាស់ mouse នៅលើធាតុ (element) ណាមួយនៅលើ Browser ។ ការប្រើប្រាស់ Mouse Events មានបួនប្រភេទដូចខាងក្រោម៖

* យើងអាចប្រើនិមិត្តសញ្ញា @ ជំនួសឲ្យ **v-on**

```
<template>
  <div v-on:mouseover="handleOver" class="box">
    <p>Mouse Over</p>
  </div>

  <div @mouseleave="handleLeave" class="box">
    <p>Mouse Leave</p>
  </div>

  <div @dblclick="hanldeDBLClick" class="box">
    <p>Double Click</p>
  </div>

  <div @click="hanldeClick" class="box">
    <p>Click</p>
  </div>
</template>

<script>
import { ref } from "@vue/reactivity";
export default {
  name: "App",
  setup() {
    const handleOver = (e) => {
      alert("I am " + e.type + " event.");
    };

    const handleLeave = (e) => {
```

```
    alert("I am " + e.type + " event.");
  };

  const handleDBLClick = (e) => {
    alert("I am " + e.type + " event.");
  };

  const handleClick = (e) => {
    alert("I am " + e.type + " event.");
  };
  return { handleOver, handleLeave, handleDBLClick, handleClick };
},
};
</script>

<style>
.box {
  background: rgb(219, 215, 215);
  width: 300px;
  height: 200;
  display: inline-block;
  text-align: center;
  cursor: pointer;
  padding-top: 100px;
  margin: 10px;
}
</style>
```

៦. List (v-for)

List មានលក្ខណៈដូចទៅនឹង Array នៅក្នុងភាសា programming ដទៃទៀតដែរ។ List ប្រើសំរាប់ផ្ទុកធាតុ (elements), ដើម្បីបង្ហាញធាតុទាំងនោះចេញមកយើងត្រូវប្រើប្រាស់ **v-for** ។

ខាងក្រោមនេះគឺជាការយកធាតុទាំងអស់នៅក្នុង Products List ចេញមកបង្ហាញ៖

```
<template>
<div>
  <h1>Product List</h1>
  <ol v-for="item in products" :key="item">
    <li>Name: {{ item.name }}</li>
    <li>Price: {{ item.price }}</li>
  </ol>
</div>
</template>
<script>
import { ref } from '@vue/reactivity';
export default {
  setup() {
    const products = ref([
```



```

    {
      name: "bag",
      price: 20,
    },
    {
      name: "shoes",
      price: 15,
    },
    {
      name: "shirt",
      price: 3,
    }
  ]
);

  return { products };
}
};
</script>
<style>
div {
  padding: 40px;
}
h1 {
  font-weight: bold;
  font-size: 50px;
  color: blue;
}
li {
  color: gray;
  font-size: 32;
  font-style: italic;
}
</style>

```

៧. Attribute Binding

Attribute Binding គឺជាការភ្ជាប់ (binding) dynamic value របស់ attribute ទៅនឹង html element ។ ដើម្បីឲ្យកាន់តែច្បាស់យើងចូលទៅមើលឧទាហរណ៍ទាំងអស់គ្នា។

ឧទាហរណ៍ទី១៖ ការប្រើប្រាស់ attribute ធម្មតា

```

<template>
  <h1>Welcome To Master Online learning</h1>
  <p>This is our <a href="https://www.facebook.com/masteritcambodia"
  target="_blank">Facebook</a> official page.</p>
</template>
<script>

```



```

export default {
};
</script>
<style>
h1,
p {
  padding: 25px;
}
h1 {
  color: blue;
  font-size: 60px;
  font-family: "Franklin Gothic Medium", "Arial Narrow", Arial, sans-serif;
  font-weight: bold;
}

p {
  font-size: 40px;
  color: gray;
}
</style>

```

ឧទាហរណ៍ទី២៖ ការប្រើប្រាស់ attribute binding

```

<template>
  <h1>Welcome To Master Online learning</h1>
  <p>This is our <a href="url" target="_blank">Facebook</a> officail page.</p>
</template>
<script>
import { ref } from "@vue/reactivity";
export default {
  setup() {
    const url = ref("https://www.facebook.com/masteritcambodia");

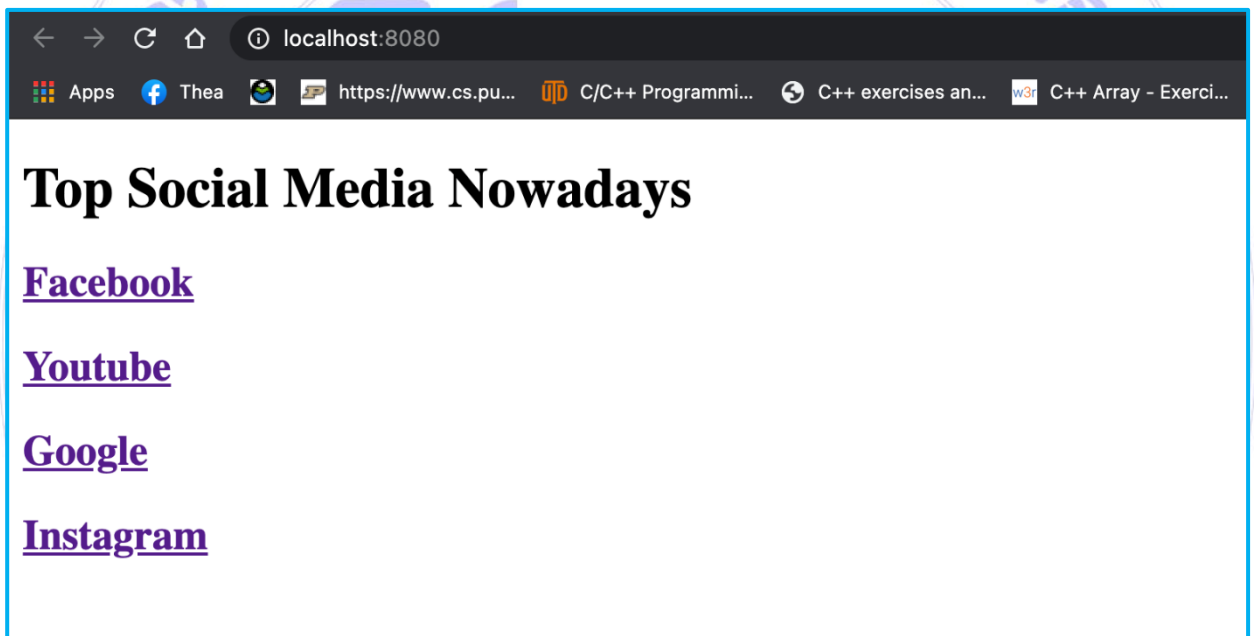
    return { url };
  },
};
</script>
<style>
h1,
p {
  padding: 25px;
}
h1 {
  color: blue;
  font-size: 60px;
  font-family: "Franklin Gothic Medium", "Arial Narrow", Arial, sans-serif;
  font-weight: bold;
}

```

```
p {
  font-size: 40px;
  color: gray;
}
</style>
```

លំហាត់

ចូរបង្កើត Social Media List មួយសំរាប់ផ្ទុក Social Media ដែលគេកំពុងពេញនិយមប្រើប្រាស់យ៉ាងតិចឲ្យបាន៤ បន្ទាប់មកប្រើប្រាស់ v-for ដើម្បីបង្ហាញ Social Media ទាំងនោះ។ នៅពេលបង្ហាញយើងត្រូវភ្ជាប់ (binding) link ទៅនឹងឈ្មោះ (brand) របស់ social media ដើម្បីយើងអាច link ទៅបាននៅពេលដែលយើងចុចលើឈ្មោះទាំងនោះ។ នៅក្នុង Social Media នីមួយៗមានធាតុ ២, ធាតុទី១មានឈ្មោះថា brand និងធាតុទី២មានឈ្មោះថា link ។ ឧទាហរណ៍៖ { brand: "Facebook", link: "https://facebook.com" }។ ខាងក្រោមនេះជាលទ្ធផលគំរូ៖



៨. Static និង Dynamic classes

នៅក្នុង Vue យើងអាចប្រើ class attribute ដើម្បីបន្ថែម style ទៅឲ្យ Html element ដើម្បីឲ្យ element មើលទៅមានភាពស្រស់ស្អាតជាងមុន។ ការប្រើប្រាស់ class មានពីរបែបគឺ static និង dynamic class ។

ឧទាហរណ៍៖

```
<template>
  <div class="container">
    <div :class="{ dynamic: true }">Home</div>
    <div class="static">Contact</div>
    <div class="static">About</div>
  </div>
</template>
<script>
  import { ref } from "@vue/reactivity";
```



```

export default {
  setup() {
  }
};
</script>
<style>
.container {
  width: 100%;
  height: 20px;
  text-align: center;
}
.static {
  color: blue;
  font-size: 40px;
  padding: 20px;
  cursor: pointer;
  display: inline-block;
}
.dynamic {
  color: pink;
  border-bottom: 2px solid pink;
  font-size: 40px;
  padding: 20px;
  cursor: pointer;
  display: inline-block;
}
</style>

```

៩. Computed Property និង V-Model

V-Model នៅក្នុង Vue គឺជា **Directive** ដែលប្រើសំរាប់ភ្ជាប់ទិន្នន័យពីរផ្លូវ (two-way binding) នៅលើ Form ភាគច្រើនយើងប្រើជាមួយ input, checkbox, select, textarea និង radio ជាដើម។

ឧទាហរណ៍៖

```

<template>
  <h1>Encourage Quote: {{ quote }}</h1>
  <div>
    <label>Quote</label>
    <input type="text" v-model="quote" />
  </div>
</template>

<script>
import { ref } from "@vue/reactivity";
export default {
  setup() {

```

```

    const quote = ref("");

    return { quote };
  },
};
</script>

<style>
h1 {
  color: gray;
  font-size: 50px;
  font-weight: bold;
}
label {
  display: block;
  font-size: 25px;
}
input {
  font-size: 25px;
  padding: 10px;
}
</style>

```

Computed Property គឺជា property មួយនៅក្នុង Vue ដែលយើងប្រើដើម្បីពណ៌នាតម្លៃមួយដែលអាស្រ័យលើតម្លៃមួយទៀត។ នៅពេលដែលយើងភ្ជាប់ទិន្នន័យ (bind-data) ទៅនឹង Computed property នៅក្នុង template, Vue នឹងកែប្រែ (update) តម្លៃនៅក្នុង Computed property នៅពេលណាដែលតម្លៃអាស្រ័យលើនោះបានផ្លាស់ប្តូរ លើសពីនេះយើងអាចប្រើវាដើម្បីដោះស្រាយបញ្ហាស្មុគស្មាញដូចទៅនឹង Methods property ដែរ។

ឧទាហរណ៍៖

```

<template>
  <h1>Greeting: {{ composing }}</h1>
  <div>
    <label>name</label>
    <input type="text" v-model="name" />
    <label>greeting</label>
    <input type="text" v-model="greeting" />
  </div>
</template>

<script>
import { ref } from "@vue/reactivity";
import { computed } from "@vue/runtime-core";
export default {
  setup() {
    const name = ref("");

```



```

    const greeting = ref("");
    const composing = computed(() => {
      return greeting.value + ", " + name.value;
    });
    return { name, greeting, composing };
  },
};
</script>

<style>
h1 {
  color: gray;
  font-size: 50px;
  font-weight: bold;
}
label {
  display: block;
  font-size: 25px;
}
input {
  font-size: 25px;
  padding: 10px;
}
</style>

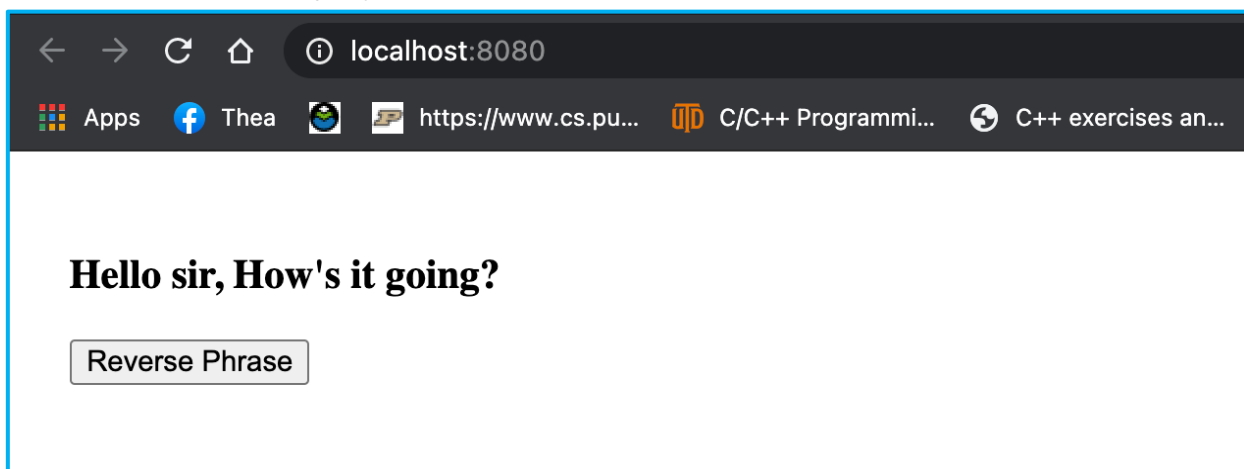
```

លំហាត់

ចូរបង្កើតប៊ូតុងមួយដែលមានឈ្មោះថា Reverse Phrase ដែលមានមុខងារសំរាប់ត្រួលបំប្លែងតាមលំដាប់បញ្ជាស។ នៅខាងលើ input tag និង ប៊ូតុង មានឃ្លាមួយថា “ Hello sir, How’s it going? ” ។

* គន្លឹះជំនួយ៖ ដើម្បីបញ្ជាសឃ្លាបានយើងត្រូវប្រើ array methods ដូចជា split, reverseនិង join ។

ខាងក្រោមនេះជាកំរូលទទួលបាន៖





90. The Different Between Computed and Methods Property

Computed property និង **Methods property** ប្រើសំរាប់ដោះស្រាយបញ្ហាស្មុគស្មាញដូចគ្នា តែចំណុចដែលខុសគ្នាគឺ **Computed property** ដឹងថា function ដែលស្ថិតនៅក្នុងវាត្រូវបានប្រើម្តងរួចហើយនោះវានឹងមិនដំណើរការម្តងទៀតទេត្រង់ចំណុចនេះមានន័យថាវាដំណើរការតែម្តងប៉ុណ្ណោះ។ ចំនែកឯ **Methods property** មិនដឹងថា function ដែលស្ថិតនៅក្នុងវាត្រូវបានប្រើរួចហើយទេដូច្នេះវាដំណើរការ function នោះរាល់ពេលដែលយើងធ្វើអន្តរកម្មទៅលើវា។

ឧទាហរណ៍៖

```
<template>
  <h1>{{ greeting }}</h1>
  <button @click="handleReversePhraseByComputed">
    Reverse Phrase By Computed Property
  </button>
  <button @click="handleReversePhraseByMethod">
    Reverse Phrase By Method Property
  </button>
</template>

<script>
import { ref } from "@vue/reactivity";
import { computed } from "@vue/runtime-core";
export default {
  setup() {
    const greeting = ref("Hello sir, How's it going?");

    const handleReversePhraseByComputed = computed(() => {
      greeting.value = greeting.value
        .split("")
        .reverse()
        .join("");
    });

    const handleReversePhraseByMethod = () => {
      greeting.value = greeting.value
```




```
    .split("")
    .reverse()
    .join("");
  };
  return {
    greeting,
    handleReversePharseByMethod,
    handleReversePharseByComputed,
  };
},
};
</script>
<style>
div {
  width: 600px;
  margin: 0 auto;
}
h1 {
  color: blue;
  font-size: 50px;
  font-weight: bold;
  font-style: italic;
}
button {
  padding: 30px;
  font-size: 25px;
}
</style>
```

មេរៀនទី៣

កម្រិតថ្នាក់មធ្យម INTERMEDIATE

១. Template Refs

Template Refs អនុញ្ញាតឱ្យយើងផ្ទុក (store) reference ទៅនឹង DOM element នៅក្នុងអញ្ញត (variable) នៅពេលដែលយើងមាន reference ហើយយើងអាចប្រើប្រាស់ JavaScript methods និង properties ដើម្បីធ្វើការដូចជា changes, classes, text content, ID styles ។ល។ តើយើងនឹងប្រើប្រាស់វាយ៉ាងដូចម្តេច? ខាងក្រោមនេះគឺជាការបង្ហាញអំពីការប្រើប្រាស់ JavaScript methods និង style នៅលើ input tag តាមរយៈ template refs ។

* ចំណាំ៖ ធាតុ (element) នៅក្នុង HTML យើងហៅថា HTML element នៅលើ Browser យើងហៅថា DOM element ។

```
<template>
  <h1 ref="h1">Template Refs</h1>
  <button @click="handleClick">Click Here</button>
</template>
<script>
import { ref } from "@vue/reactivity";
export default {
  setup() {
    const h1 = ref("");
    const handleClick = () => {
      h1.value.classList.add("giant");
      h1.value.innerHTML = "I am Template Refs.";
      console.log(h1.value);
    };
    return { h1, handleClick };
  },
};
</script>

<style>
h1 {
  font-size: 50px;
  color: blue;
  font-style: italic;
}
button {
  padding: 20px;
  font-size: 20px;
}
.giant {
  font-size: 90px;
  color: gray;
}
</style>
```



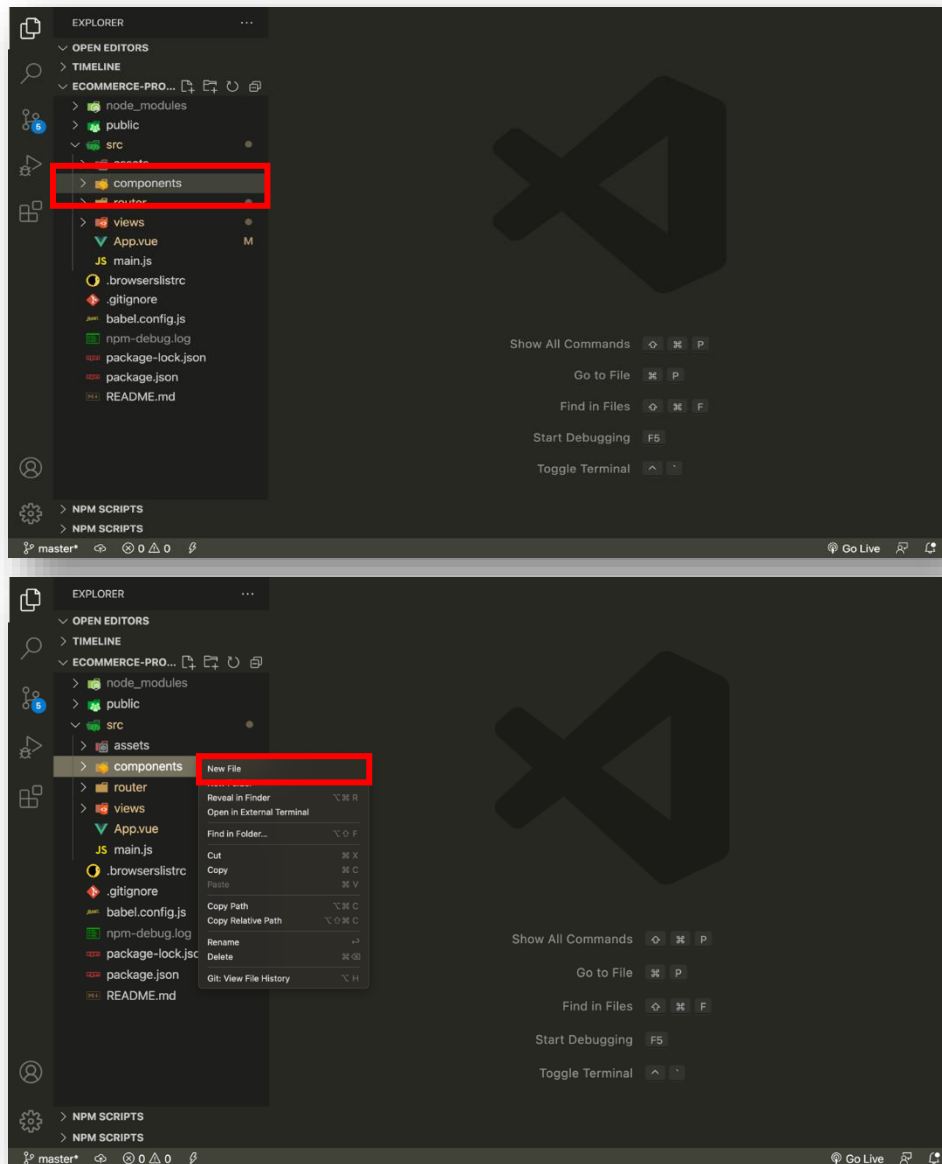
២. Component និង Multi Components

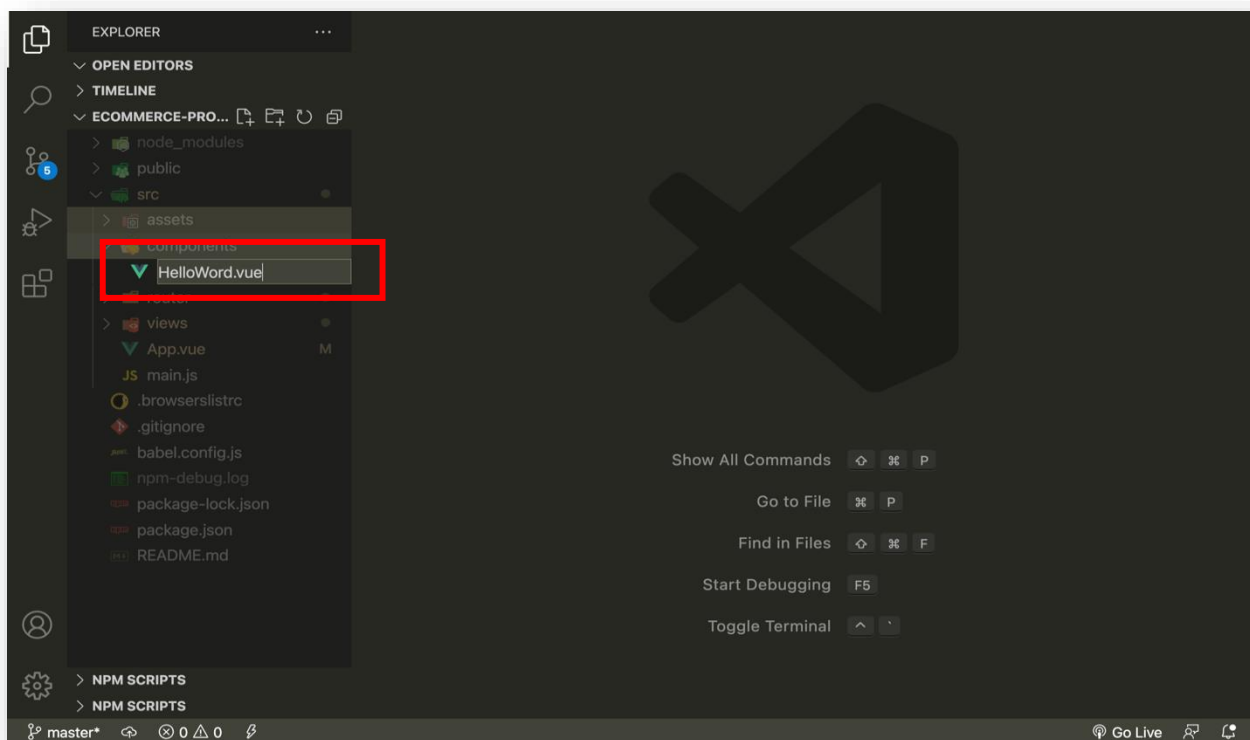
Component គឺជាផ្នែកមួយដ៏មានឥទ្ធិពលរបស់ Vue.js ។ យើងបង្កើត Component តែម្តង ហើយហៅវាប្រើបានច្រើនដងនិងច្រើនកន្លែង(reusable) នៅក្នុង HTML ។ ចំណែក **Multi Components** គឺយើងប្រើប្រាស់ Components ច្រើនក្នុង page តែមួយ។ ច្បាប់នៃការដាក់ឈ្មោះ Component, Component ត្រូវផ្តើមដោយអក្សរធំមួយតួដំបូងនិងត្រូវចាប់ពីពាក្យឡើងទៅដើម្បីកុំឲ្យច្រឡំជាមួយ HTML tag ពិសេសជាងនេះគឺ Component extension ត្រូវតែ .vue extension ។

ឧទាហរណ៍នៃការបង្កើត Component និងការហៅប្រើនៅក្នុង Main Page ឬ Root Page in Vue project៖ HelloWorld.vue

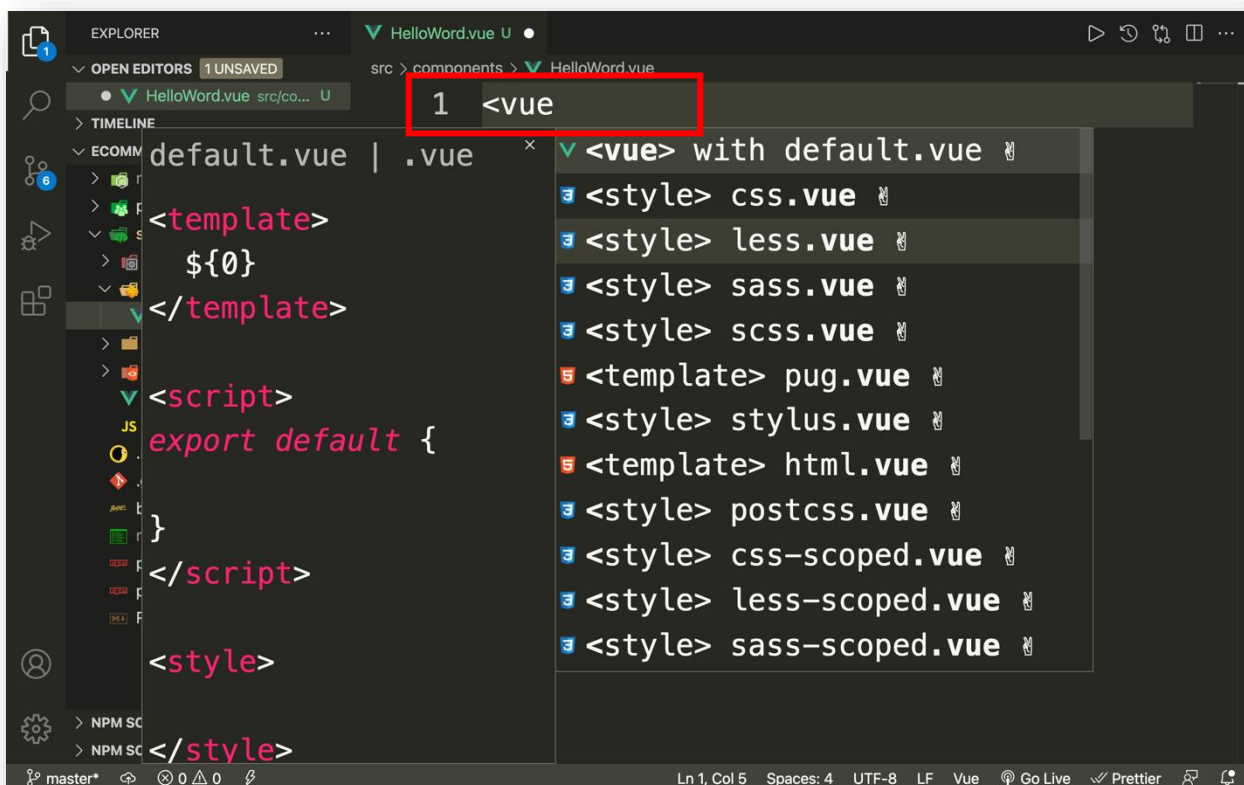
មុនដំបូងយើងយក Cursor ទៅដាក់លើ components folder ដែលស្ថិតនៅក្នុង src folder រួចចុច mouse ស្តាំហើយជ្រើសរើសពាក្យថា New File បន្ទាប់មកដាក់ឈ្មោះថា HelloWorld.vue ។

ខាងក្រោមនេះគឺជាគំរូ៖





បន្ទាប់មកវាយបញ្ចូលពាក្យ `<vue` ហើយចុច tab នោះយើងនឹងបាន barebones ស្រាប់មួយដូចរូបបន្ទាប់ខាងក្រោម។





```
1 <template>
2 |
3 </template>
4
5 <script>
6 export default {
7
8 }
9 </script>
10
11 <style>
12
13 </style>
```

```
1 <template>
2 <h2>Hello I am from HelloWorld Component.</h2>
3 </template>
4
5 <script>
6 export default {
7
8 }
9 </script>
10
11 <style>
12
13 </style>
```

ខាងក្រោមនេះគឺជាការហៅ Component មកប្រើនៅក្នុង Root Page ៖

```

1 <template>
2   <div>
3     <h3>I am main page in Vue Project.</h3>
4     <HelloWorld />
5   </div>
6 </template>
7
8 <script>
9   import HelloWorld from "@components/HelloWorld.vue";
10  export default {
11    components: {
12      HelloWorld,
13    },
14  };
15 </script>
16
17 <style></style>
18

```

ឧទាហរណ៍នៃការប្រើ Multi Components ៖ យើងគ្រាន់តែបង្កើត Component មួយទៀត ដោយដាក់ឈ្មោះថា Introduction.vue ដោយការបង្កើតយកលំនាំតាមខាងលើ ។

```

<template>
  <h3>I am John Cena, I am an American professional wrestler.</h3>
  <h3>I was born in April 23, 1877.</h3>
</template>

<script>
export default ៖;
</script>

<style>
h3 {
  font-size: 40px;
  color: plum;
}
</style>

```

បន្ទាប់មកហៅប្រើនៅក្នុង Root Page ៖

```

<template>
  <div>
    <h1 ref="h1">Template Refs</h1>
    <button @click="handleClick">Click Here</button>

```



```

</div>
<HelloWorld />
<Introduction />
</template>

<script>
import { ref } from "@vue/reactivity";
import HelloWorld from "@components/HelloWorld.vue";
import Introduction from "@components/Introduction.vue";
export default {
  components: {
    HelloWorld,
    Introduction,
  },
  setup() {
    const h1 = ref("");

    const handleClick = () => {
      h1.value.classList.add("giant");
      h1.value.innerHTML = "I am Template Refs.";
      console.log(h1.value);
    };

    return { h1, handleClick };
  },
};
</script>

<style>
h1 {
  font-size: 50px;
  color: blue;
  font-style: italic;
}
button {
  padding: 20px;
  font-size: 20px;
}
.giant {
  font-size: 90px;
  color: gray;
}
</style>

```

៣. Dynamic Components

យើងប្រើ **Dynamic Components** ដើម្បីផ្លាស់ប្តូរ Component នៅលើ page ដោយប្រើប្រាស់ **component tag** និង **is** attribute ។ ខាងក្រោមនេះគឺជាឧទាហរណ៍នៃការប្រើប្រាស់ ។


```

<template>
  <div>
    <h1>Dynamic Components</h1>
    <component :is="currentComponent" />
    <button @click="handleSwitch('HelloWorld')">Hello World</button>
    <button @click="handleSwitch('Introduction')">Introduction</button>
  </div>
</template>

<script>
import { ref } from "@vue/reactivity";
import HelloWorld from "@components/HelloWorld.vue";
import Introduction from "@components/Introduction.vue";
export default {
  setup() {
    const currentComponent = ref("");

    const handleSwitch = (component) => {
      currentComponent.value = component;
    };
    return { currentComponent, handleSwitch };
  },
};
</script>

<style>
div {
  width: 700px;
  margin: 0 auto;
}
h1 {
  color: blue;
  font-size: 50px;
}
button {
  font-size: 30px;
  padding: 20px;
  margin: 0 10px;
}
</style>

```

៤. Component Styles និង Global Styles

Component Styles គឺជាការលេង style នៅលើ component នីមួយៗដោយមិនឲ្យប៉ះពាល់ទៅលើ Component ដទៃទៀតដោយប្រើប្រាស់ scoped attribute នៅក្នុង style tag ។

Global Styles គឺជាការលេង style នៅលើ component និង page ទាំងអស់នៅក្នុង Vue Project ដោយសរសេរតែមួយកន្លែងវានឹង apply style ដោយស្វ័យប្រវត្តិទៅលើ page និង component ទាំងអស់។

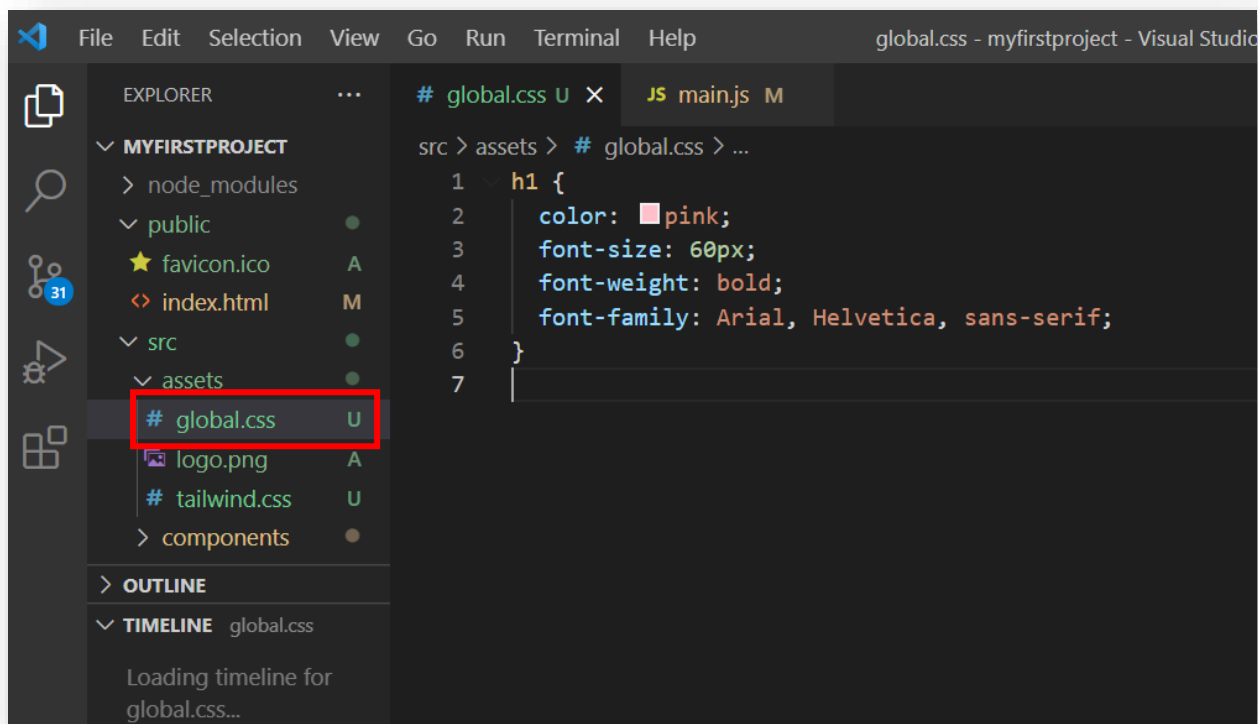
ឧទាហរណ៍ Component styles នៅក្នុង Introduction.vue ៖

```
<template>
  <h1>I am John Cena, I am an American professional wrestler.</h1>
  <h1>I was born in April 23, 1877.</h1>
</template>

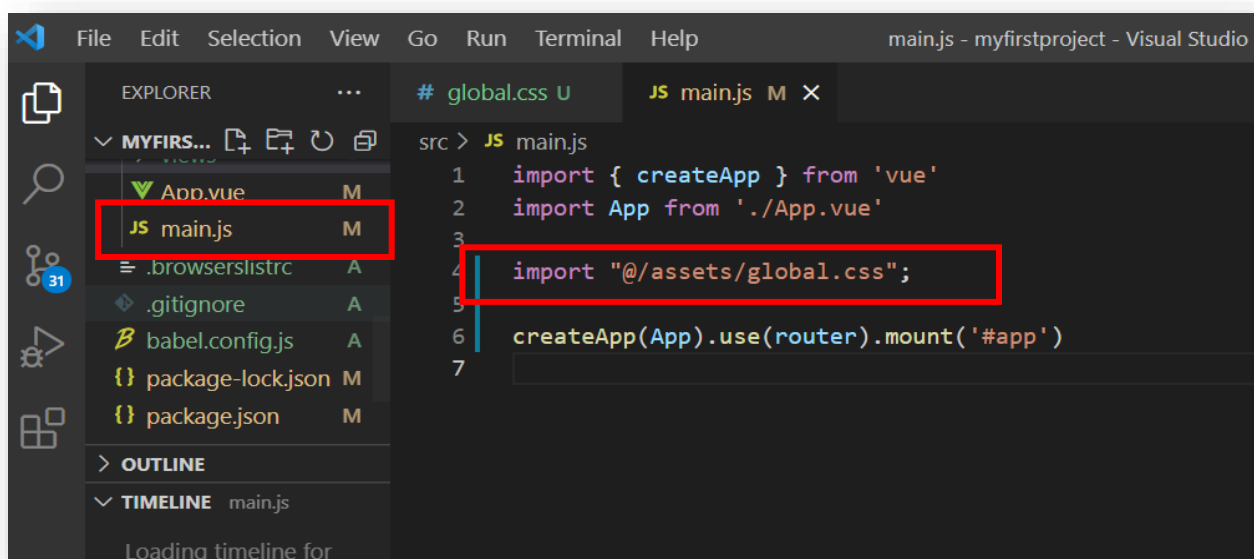
<script>
export default {};
</script>

<style scoped>
h1 {
  color: red;
  font-size: 60px;
}
</style>
```

ឧទាហរណ៍ Global styles និង ទីតាំងរបស់វា៖ ជាទូទៅយើងបង្កើត Global styles file ដោយដាក់ឈ្មោះថា style.css, main.css, ឬ global.css ហើយទីតាំងរបស់វាគឺស្ថិតនៅក្នុង assets folder ។



បន្ទាប់ពីបង្កើតបានហើយត្រូវ import ទៅក្នុង file មួយមានឈ្មោះថា main.js ជាការស្រេច។



៥. Passing Data with Props

Props គឺជា property របស់ Component ដែល props មានតួនាទីសំរាប់ទទួលទិន្នន័យពី Parent Component ហើយបញ្ជូនទៅឲ្យ child property ដែលនៅក្នុង Component នោះ។ មូលហេតុដែលយើងគួរប្រើ props ពីព្រោះយើងអាចទទួលបាន Dynamic ទិន្នន័យ (data) ។ ឥឡូវនេះយើងទៅមើលឧទាហរណ៍ខាងក្រោមទាំងអស់គ្នា។

នៅក្នុង App.vue

```
<template>
  <div>
    <h1>Props</h1>
    <Introduction :bio="bio"/>
  </div>
</template>

<script>
import { ref } from "@vue/reactivity";
import Introduction from "@components/Introduction.vue";
export default {
  components: {
    Introduction,
  },
  setup() {
    const bio = ref({
      name: "John Cena",
      dob: "April 23, 1877",
      title: "wrestler",
    });
    return { bio };
  }
}
```



```

    },
  };
</script>
<style></style>

```

នៅក្នុង Introduction.vue

```

<template>
  <h1>I am {{ bio.name }}, I am an American professional {{ bio.title }}.</h1>
  <h1>I was born in {{ bio.dob }}.</h1>
</template>

<script>
export default {
  props: ["bio"],
}
</script>

<style scoped>

</style>

```

៦. Emitting Events

Emitting events គឺការបញ្ជូនទិន្នន័យពី Child property មក Parent Component វិញតាម

វិធី: emit function ។

ខាងក្រោមនេះគឺជាឧទាហរណ៍ដែលលោកគ្រូប្រើ emit function ដើម្បីបញ្ជូនទិន្នន័យមកឲ្យ Parent Component វិញ។

នៅក្នុង App.vue

```

<template>
  <div>
    <h1>Props</h1>
    <Introduction :bio="bio" @greeting="handleGreeting" />
  </div>
</template>

<script>
import { ref } from "@vue/reactivity";
import Introduction from "@components/Introduction.vue";
export default {
  components: {
    Introduction,
  },
  setup() {
    const bio = ref({
      name: "John Cena",
      dob: "April 23, 1877",

```

```

    title: "wrestler",
  });

  const handleGreeting = (e) => {
    alert(e);
  };

  return { bio, handleGreeting };
},
};
</script>

<style></style>

```

នៅក្នុង Introduction.vue

```

<template>
  <h1>I am {{ bio.name }}, I am an American professional {{ bio.title }}.</h1>
  <h1>I was born in {{ bio.dob }}.</h1>
</template>

<script>
export default {
  props: ["bio"],
  setup(props, { emit }) {
    emit("greeting", "I am child property, I'm from Introduction Component!")
  }
}
</script>

<style scoped>
</style>

```

៧. Event Modifiers

Event Modifiers គឺជា property របស់ Event handlers ឬ Directive ដែលមានតួនាទីសំរាប់ផ្លាស់ប្តូរលក្ខណៈដើមរបស់ Directive ។

Event Modifiers មានដូចជា ៖

```

.shift
.prevent
.capture
.self
.once

```

ឧទាហរណ៍៖

```

<template>
  <div>

```

```
<h3>Event Modifiers</h3>
<button @click.shift="handleToggle">Toggle</button>
<component :is="currentComponent" />
</div>
</template>

<script>
import { ref } from "@vue/reactivity";
import Introduction from "@components/Introduction.vue";
import HelloWorld from "@components/HelloWorld.vue";
export default {
  components: {
    Introduction,
    HelloWorld,
  },
  setup() {
    const currentComponent = ref("");

    const handleToggle = () => {
      currentComponent.value = "HelloWorld";
    };
    return { handleToggle, currentComponent };
  },
};
</script>

<style>
h3 {
  color: blue;
  font-size: 40px;
}
button {
  padding: 20px;
  font-size: 20px;
}
</style>
```

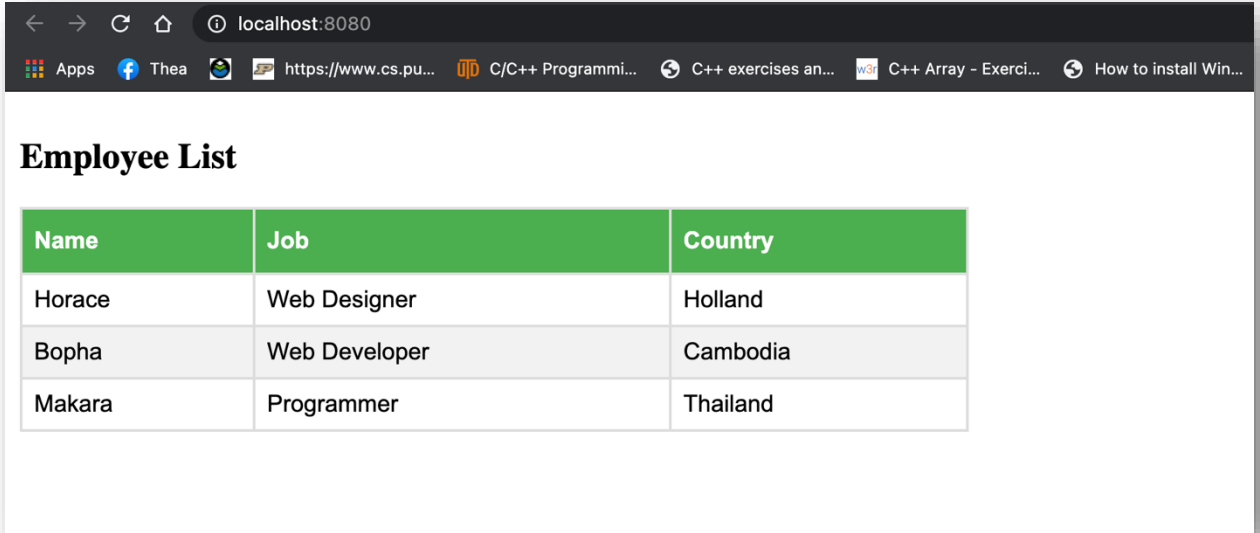
```
<div>
  <h3>Event Modifiers</h3>
  <button @click.shift="handleToggle">Toggle</button>
  <component :is="currentComponent" />
</div>
```

ទម្រង់នៃការប្រើប្រាស់៖ Directive + និមិត្តសញ្ញា dot (.) + Modifiers

លំហាត់

ចូរបង្កើត Component មួយមានឈ្មោះថា ListEmployee ដែលសំរាប់បង្ហាញឈ្មោះបុគ្គលិក តួនាទី និង ប្រទេសដែល Component នោះទទួលទិន្នន័យតាមរយៈ props ។ លើសពីនេះយើងត្រូវតែបង្ហាញទិន្នន័យជា table ។

ខាងក្រោមនេះជាលទ្ធផលគំរូ៖



The screenshot shows a web browser window with the address bar at localhost:8080. The page title is 'Employee List'. Below the title is a table with the following data:

Name	Job	Country
Horace	Web Designer	Holland
Bopha	Web Developer	Cambodia
Makara	Programmer	Thailand



មេរៀនទី៤

ទម្រង់បញ្ចូលទិន្នន័យ FORM

១. ការប្រើប្រាស់និងការណែនាំ

Form គឺជាទម្រង់សំរាប់បញ្ចូលទិន្នន័យមួយប្រភេទនៅលើ Web Page ដែលអនុញ្ញាតឲ្យអ្នកប្រើប្រាស់បញ្ចូលទិន្នន័យឬព័ត៌មានរបស់ពួកគេ។ គោលបំណងនៃការប្រើប្រាស់ Form គឺដើម្បីបញ្ជូនទិន្នន័យទៅរក្សាទុកនៅ Server ។

ឥឡូវយើងទៅមើលឧទាហរណ៍នៃការបង្កើត Sign Up Form និង ការភ្ជាប់ទិន្នន័យពីរផ្លូវនៅលើ Form ក្នុង page SignupForm.vue ទាំងអស់គ្នា ។

```
<template>
  <div>
    <form>
      <h3>Signup Form</h3>
      <label for="">Username</label>
      <input type="text" v-model="username" />
      <label for="">Email</label>
      <input type="email" v-model="email" />
      <label for="">Password</label>
      <input type="password" v-model="password" />
    </form>
    <p>Username: {{ username }}</p>
    <p>Email: {{ email }}</p>
    <p>Password: {{ password }}</p>
  </div>
</template>

<script>
import { ref } from "@vue/reactivity";
export default {
  setup() {
    const username = ref("");
    const email = ref("");
    const password = ref("");

    return { username, email, password };
  },
};
</script>

<style>
div {
  width: 900px;
  margin: 30px auto;
}
p {
  font-size: 30px;
}
```



```

    color: gray;
    font-style: italic;
  }
  form {
    max-width: 600px;
    background-color: white;
    padding: 40px;
    border-radius: 10px;
    text-align: left;
  }
  h3 {
    font-size: 40px;
    color: blue;
    text-align: center;
  }
  label {
    display: inline-block;
    color: #aaa;
    margin: 25px 0 15px;
    font-size: 20px;
    letter-spacing: 1px;
    text-transform: uppercase;
  }
  input {
    display: block;
    color: #555;
    padding: 10px 6px;
    width: 100%;
    border: none;
    border-bottom: 1px solid #ddd;
  }
</style>

```

២. Select Fields, Checkbox, និង Keyboard Events

យើងប្រើ [Select Fields](#) ដើម្បីបង្កើត drop-down list ចំនែកឯ checkbox សំរាប់ជ្រើសរើស បានច្រើនជម្រើស។ [Keyboard Event](#) កើតឡើងនៅពេលចុច key ណាមួយនៅលើ Keyboard ។ ឥឡូវយើងបន្ថែម Select, Checkbox, និង Keyboard Event ទៅក្នុងឧទាហរណ៍ខាងលើ៖

```

<template>
  <div>
    <form>
      <h3>Signup Form</h3>
      <label for="">Username</label>
      <input type="text" v-model="username" />
      <label for="">Email</label>
      <input type="email" v-model="email" />
      <label for="">Password</label>
    </form>
  </div>
</template>

```



```

    <input type="password" v-model="password" />
    <select v-model="role">
      <option value="programmer">Programmer</option>
      <option value="developer">Developer</option>
      <option value="designer">Designer</option>
    </select>
    <label for="">Skill</label>
    <input type="text" v-model="skill" @keypress.space="handleAddSkill" />
    <span>{{ skills }}</span>
  </div>
  <input type="checkbox" v-model="terms" />
  <label for="">AGREE TERMS AND CONDITIONS</label>
</div>
</form>
<p>Username: {{ username }}</p>
<p>Email: {{ email }}</p>
<p>Password: {{ password }}</p>
<p>Role: {{ role }}</p>
<p>Skills: {{ skills }}</p>
<p>Terms: {{ terms }}</p>
</div>
</template>

<script>
import { ref } from "@vue/reactivity";
export default {
  setup() {
    const username = ref("");
    const email = ref("");
    const password = ref("");
    const role = ref("");
    const skill = ref("");
    const skills = ref([]);
    const terms = ref(false);

    const handleAddSkill = () => {
      skills.value.push(skill.value);
      skill.value = "";
    };

    return {
      username,
      email,
      password,
      role,
      skill,
      skills,
      terms,
      handleAddSkill,
    };
  }
};

```

```

    },
  };
</script>

<style>
div {
  width: 900px;
  margin: 30px auto;
}
p {
  font-size: 30px;
  color: gray;
  font-style: italic;
}
form {
  max-width: 600px;
  background-color: white;
  padding: 40px;
  border-radius: 10px;
  text-align: left;
}
h3 {
  font-size: 40px;
  color: blue;
  text-align: center;
}
label {
  display: inline-block;
  color: #aaa;
  margin: 25px 0 15px;
  font-size: 20px;
  letter-spacing: 1px;
  text-transform: uppercase;
}
input,
select {
  display: block;
  color: #555;
  padding: 10px 6px;
  width: 100%;
  border: none;
  border-bottom: 1px solid #ddd;
}
span {
  margin: 10px 0;
  font-size: 30px;
  color: blue;
}
input[type="checkbox"] {
  display: inline-block;

```



```
margin: 0 20px 0 0;  
width: 20px;  
}  
</style>
```

លំហាត់

ចូរបង្កើតប៊ូតុង Submit មួយសំរាប់បញ្ជូនទិន្នន័យនៅលើ Form ដែលយើងបង្កើតខាងលើទៅ Server តែនៅក្នុងលំហាត់នេះយើងគ្រាន់តែប្រើវាសំរាប់យកទិន្នន័យមកបង្ហាញតាមរយៈ Console សិនប៉ុណ្ណោះ។

មេរៀនទី៥

VUE ROUTER

១. ហេតុអ្វីប្រើ Vue Router និងការបញ្ចូលវានៅក្នុង Project

Vue Router គឺជា router ផ្លូវការមួយរបស់ Vue.js ដែលយើងប្រើសំរាប់ឆ្លង (Navigate) ពី Page មួយទៅ Page មួយផ្សេងទៀត។ ដើម្បីបញ្ចូល Vue Router ទៅក្នុង Project របស់យើងមុន ដំបូងយើងចូលទៅកាន់ Terminal ឬ Command Prompt រួចឈរលើ Folder Project បន្ទាប់មក វាយបញ្ចូលពាក្យបញ្ជា `vue add router` បន្ទាប់មកគេនឹងឲ្យយើងជ្រើសរើស (yes/no), យើងវាយបញ្ចូល (y ឬ yes) ហើយចុច enter បន្ទាប់មកវាយបញ្ចូល (y ឬ yes) ម្តងទៀតជាការស្រេច។ ខាងក្រោមនេះជាគំរូ ៖

```
thearith@Phengs-MacBook-Pro ~/Desktop/ecommerce-project$ clear
thearith@Phengs-MacBook-Pro ~/Desktop/ecommerce-project$ vue add router
WARN There are uncommitted changes in the current repository, it's recommended to commit or stash them first.
? Still proceed? Yes
Installing @vue/cli-plugin-router...
+ @vue/cli-plugin-router@4.5.13
updated 1 package and audited 1244 packages in 10.194s
69 packages are looking for funding
  run `npm fund` for details
found 109 vulnerabilities (108 moderate, 1 high)
  run `npm audit fix` to fix them, or `npm audit` for details
✓ Successfully installed plugin: @vue/cli-plugin-router
? Use history mode for router? (Requires proper server setup for index fallback in production) Yes
Invoking generator for @vue/cli-plugin-router...
✓ Successfully invoked generator for plugin: @vue/cli-plugin-router
thearith@Phengs-MacBook-Pro ~/Desktop/ecommerce-project$
```

២. Dynamic និង Static Router Link

Vue Router បែងចែកជាពីរគឺ static router link និង dynamic router link ។ static router link យើងប្រើប្រាស់ដើម្បី navigate តាមរយៈតម្លៃរបស់ path ចំនែក dynamic router link យើងប្រើប្រាស់ដើម្បី navigate ដូចគ្នាប៉ុន្តែតាមរយៈ name ហើយគេពេញនិយមប្រើប្រាស់ dynamic router link ជាង។ តើអ្វីជា path និង name ដែលយើងបានលើកឡើងខាងលើ? Path ប្រៀបដូចជាផ្លូវ (url) មួយដែលនាំយើងទៅកាន់ Page ឬ ទីតាំងណាមួយនៅលើ Website ដែលយើងចង់ទៅ ចំនែក Name គឺជាឈ្មោះរបស់ Page ឬ ទីតាំងណាមួយនៅលើ Website ជាទូទៅ path និង name មានទំនាក់ទំនងនឹងគ្នា ។

ឧទាហរណ៍៖

```
<template>
  <div id="nav">
    <router-link to="/">Home</router-link> |
```



```

    <router-link to="/about">About</router-link>
  </div>
</router-view>
</template>

```

```

<style>
#app {
  font-family: Avenir, Helvetica, Arial, sans-serif;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
  text-align: center;
  color: #2c3e50;
}
#nav {
  padding: 30px;
}
#nav a {
  font-weight: bold;
  color: #2c3e50;
}

#nav a.router-link-exact-active {
  color: #42b983;
}
</style>

```

* ដើម្បីប្រើប្រាស់ router link ទាំងពីរប្រភេទខាងលើបានយើងត្រូវចូលទៅកាន់ index.js file ដែលទីតាំងរបស់ file ស្ថិតនៅក្នុង folder router បន្ទាប់មកសរសេរកូដដូចខាងក្រោម ៖

```

import { createRouter, createWebHistory } from 'vue-router'
import Home from '../views/Home.vue'
const routes = [
  {
    path: '/',
    name: 'Home',
    component: Home
  },
  {
    path: '/about',
    name: 'About',
    // route level code-splitting
    // this generates a separate chunk (about.[hash].js) for this route
    // which is lazy-loaded when the route is visited.
    component: () => import(/* webpackChunkName: "about" */ '../views/About
.vue')
  }
]

```



```
const router = createRouter({
  history: createWebHistory(process.env.BASE_URL),
  routes
})

export default router
```

៣. Route Parameters

Route parameters គឺជាផ្នែកនៃ url ដែលនឹងត្រូវផ្លាស់ប្តូរអាស្រ័យថាតើយើងចង់ឲ្យអ្វីបង្ហាញចេញមក។ ឧទាហរណ៍ថាយើងចង់ឲ្យបង្ហាញព័ត៌មានរបស់បុគ្គលិកទី១ អញ្ចឹងយើងគួរតែចូលទៅកាន់ path (/employees/staff_id) តែប្រសិនបើយើងចង់ឲ្យបង្ហាញព័ត៌មានរបស់បុគ្គលិកទី២ យើងចូលទៅកាន់ path (/employees/ staff_id) ។

ឧទាហរណ៍ ៖

នៅក្នុង App.vue

```
<template>
  <div id="nav">
    <router-link to="/">Home</router-link> |
    <router-link to="/about">About</router-link>
    <router-link :to="{ name: 'Employee' }">Employee</router-link>
  </div>
  <router-view />
</template>

<style>
#app {
  font-family: Avenir, Helvetica, Arial, sans-serif;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
  text-align: center;
  color: #2c3e50;
}

#nav {
  padding: 30px;
}

#nav a {
  font-weight: bold;
  color: #2c3e50;
}

#nav a.router-link-exact-active {
  color: #42b983;
}
```



</style>

នៅក្នុង index.js

```
import { createRouter, createWebHistory } from 'vue-router'
import Home from '../views/Home.vue'
import Employee from '../views/Employee.vue'
import Staff from '../views/Staff.vue'

const routes = [
  {
    path: '/',
    name: 'Home',
    component: Home
  },
  {
    path: '/about',
    name: 'About',
    // route level code-splitting
    // this generates a separate chunk (about.[hash].js) for this route
    // which is lazy-loaded when the route is visited.
    component: () => import(/* webpackChunkName: "about" */ '../views/About
.vue')
  },
  {
    path: '/employee',
    name: "Employee",
    component: Employee
  },
  {
    path: '/employee/:id',
    name: "Staff",
    component: Staff
  }
]

const router = createRouter({
  history: createWebHistory(process.env.BASE_URL),
  routes
})

export default router
```




នៅក្នុង Employees.vue

```

<template>
  <div>
    <ul v-for="staff in employee" :key="staff.id">
      <router-link :to="{ name: 'Staff', params: {id: staff.id}}">
        <li>{{ staff.name }}</li>
      </router-link>
    </ul>
  </div>
</template>

<script>
import { ref } from '@vue/reactivity'
export default {
  setup() {
    const employee = ref([
      {
        id: "emp_001",
        name: "Bopha"
      },
      {
        id: "emp_002",
        name: "Rothana"
      }
    ])
    return { employee };
  }
}
</script>

<style>

</style>

```

នៅក្នុង Staff.vue

```

<template>
  <div>
    <h1>Staff ID: {{ $route.params.id }}</h1>
  </div>
</template>

<script>
export default {
}
</script>

<style>

</style>

```

ជំពូកទី២

TAILWINDCSS

មេរៀនទី១

ការនិយមន័យ និងដំឡើង

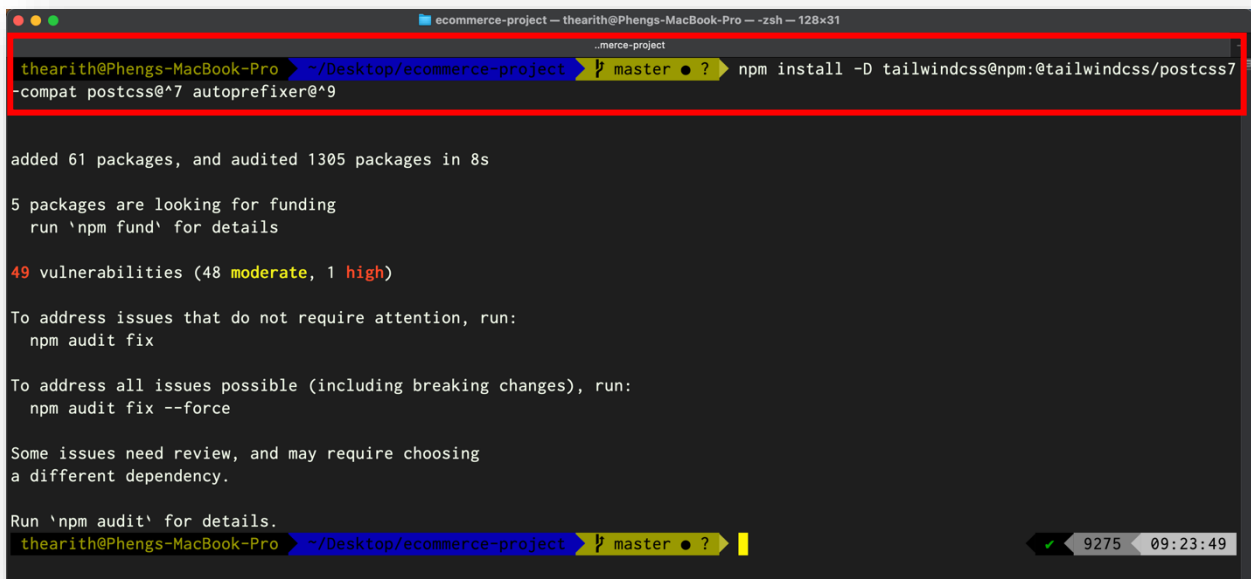
១. តើអ្វីទៅជា Tailwindcss ?

Tailwind CSS គឺជា CSS Framework ដែលផ្តល់ឲ្យយើងនូវ CSS Classes និង Tools ដែលមានភាពងាយស្រួលក្នុងការលេង Style នៅលើ Websites ។ លើសពីនេះយើងអាច Resuable CSS បានថែមទៀត។

២. ការដំឡើង

ដើម្បីធ្វើការដំឡើងបានមុនដំបូងយើងចូលទៅកាន់ Terminal ឬ Command Prompt រួចឈរនៅលើ Project ហើយវាយបញ្ចូលពាក្យបញ្ជា៖

```
npm install -D tailwindcss@npm:@tailwindcss/postcss7-compat postcss@^7 autoprefixer@^9 ។
```



```
thearith@Phengs-MacBook-Pro ~ % npm install -D tailwindcss@npm:@tailwindcss/postcss7-compat postcss@^7 autoprefixer@^9

added 61 packages, and audited 1305 packages in 8s

5 packages are looking for funding
  run `npm fund` for details

49 vulnerabilities (48 moderate, 1 high)

To address issues that do not require attention, run:
  npm audit fix

To address all issues possible (including breaking changes), run:
  npm audit fix --force

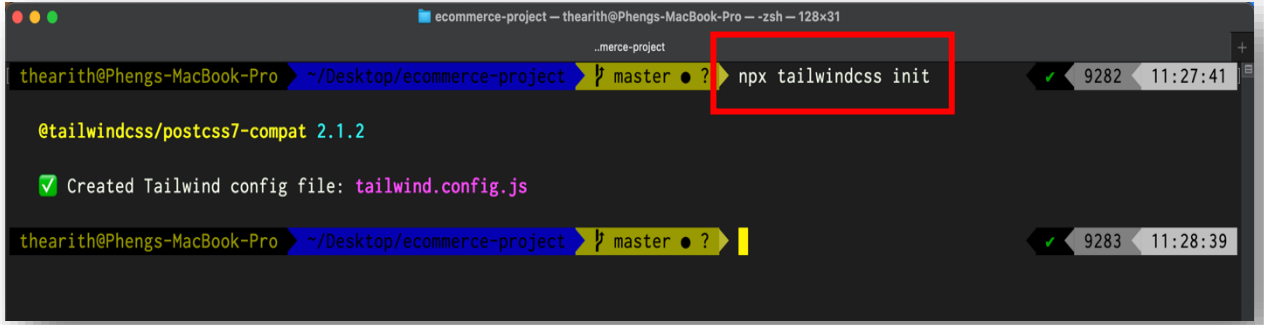
Some issues need review, and may require choosing
a different dependency.

Run `npm audit` for details.
```

៣. Configuration

បន្ទាប់ពីដំឡើងហើយយើងត្រូវតែទៅកំណត់ (config) ឲ្យភាសា Tailwindcss និង VueJS ស្គាល់គ្នាសិនទើបអាចប្រើប្រាស់បាន។ ក្នុងការកំណត់លោកគ្រូបានបែងចែកវាជា៣តំណក់កាល៖

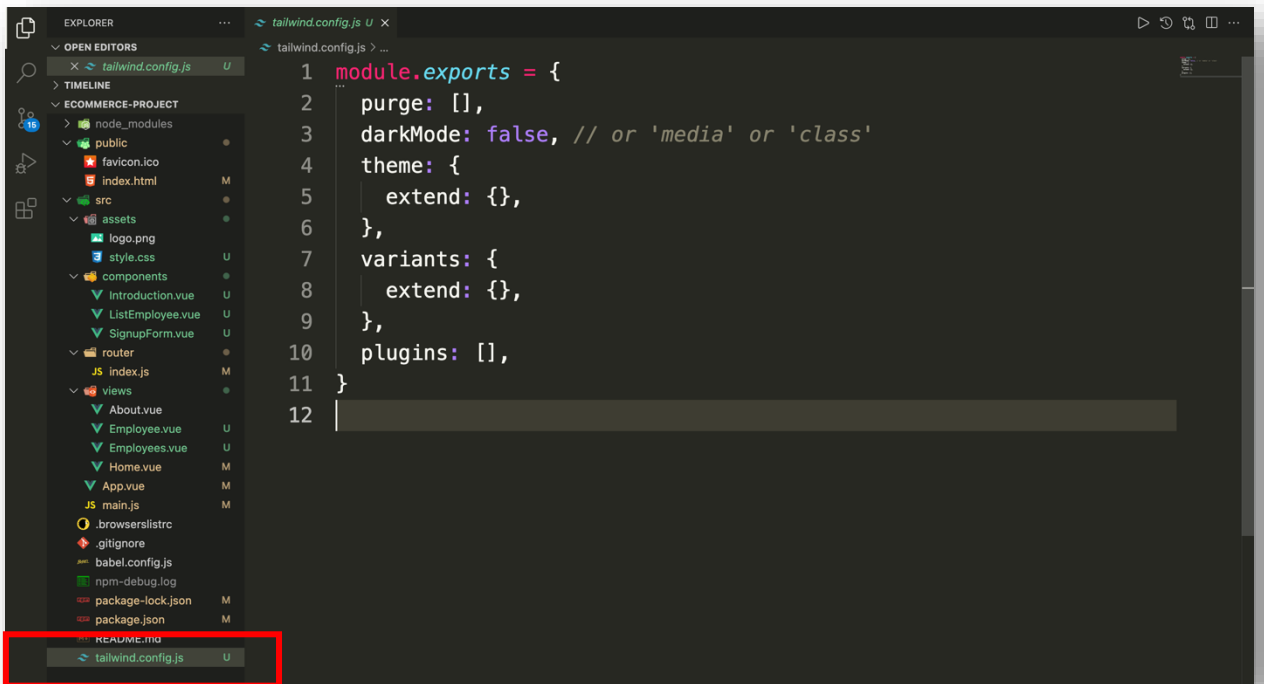
តំណក់កាលទី១៖ បង្កើត file មួយដែលមានឈ្មោះថា `tailwind.config.js` ដោយប្រើប្រាស់ពាក្យបញ្ជា `npx tailwindcss init`



```

ecommerce-project — thearith@Phengs-MacBook-Pro — zsh — 128x31
thearith@Phengs-MacBook-Pro ~ /Desktop/ecommerce-project  master ● ? npx tailwindcss init
@tailwindcss/postcss7-compat 2.1.2
✓ Created Tailwind config file: tailwind.config.js
thearith@Phengs-MacBook-Pro ~ /Desktop/ecommerce-project  master ● ?
  
```

បន្ទាប់មកយើងនឹងបាន file មួយឈ្មោះថា `tailwind.config.js` នៅក្នុង project របស់យើងដូចរូបខាងក្រោម៖



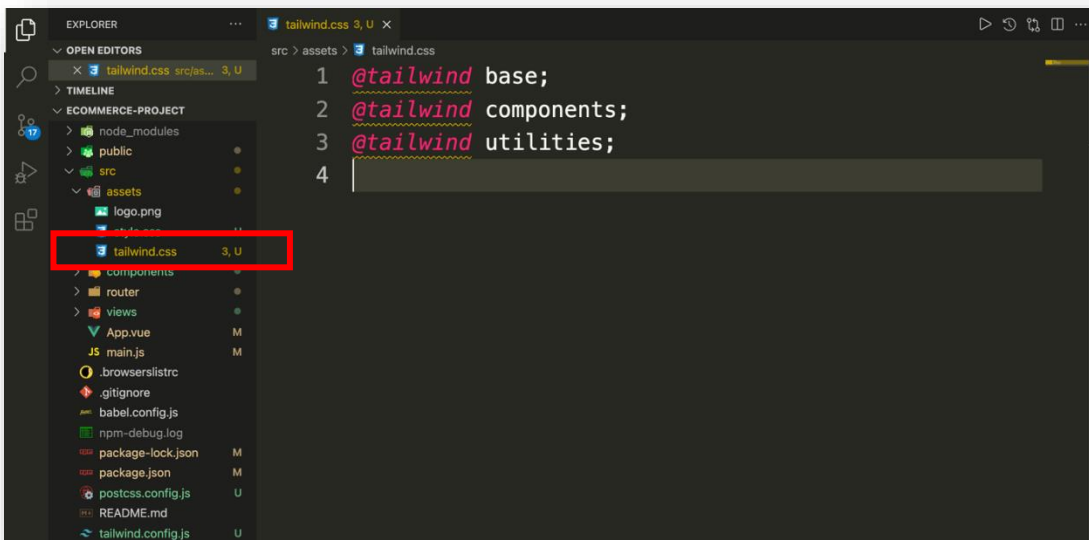
```

EXPLORER
  OPEN EDITORS
    tailwind.config.js
  TIMELINE
  ECOMMERCE-PROJECT
    node_modules
    public
      favicon.ico
      index.html
    src
      assets
        logo.png
        style.css
      components
        Introduction.vue
        ListEmployee.vue
        SignupForm.vue
      router
        index.js
      views
        About.vue
        Employee.vue
        Employees.vue
        Home.vue
        App.vue
        main.js
    .browserlistrc
    .gitignore
    babel.config.js
    npm-debug.log
    package-lock.json
    package.json
    README.md
    tailwind.config.js
  Editor
    tailwind.config.js
      1 module.exports = {
      2   purge: [],
      3   darkMode: false, // or 'media' or 'class'
      4   theme: {
      5     extend: {},
      6   },
      7   variants: {
      8     extend: {},
      9   },
      10  plugins: [],
      11 }
      12
  
```

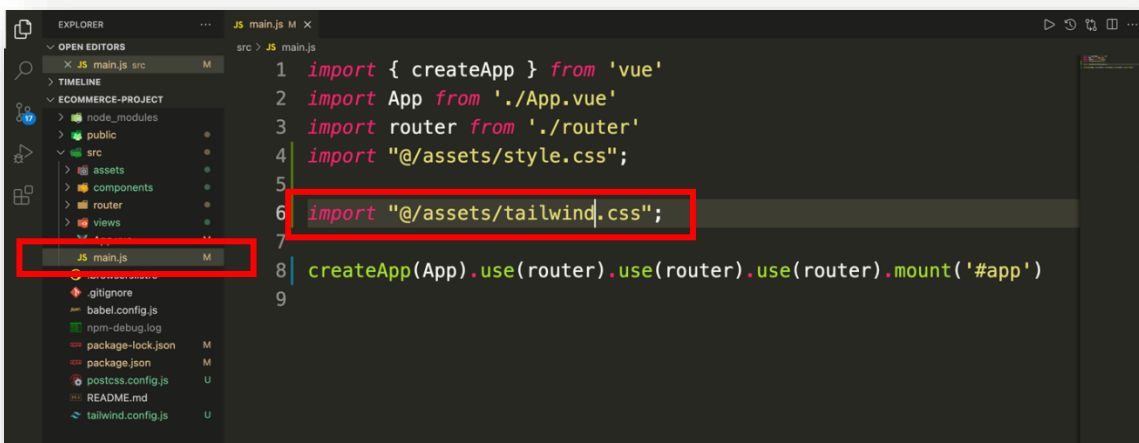
តំណក់កាលទី២៖ យើងបង្កើត file មួយទៀតដែលមានឈ្មោះថា `postcss.config.js` ដោយចូលទៅបង្កើតផ្ទាល់នៅ Project ហើយនៅក្នុង file នោះមានកូដដូចខាងក្រោម៖



តំណក់កាលទី៣៖ យើងបង្កើត file មួយទៀតដែលមានឈ្មោះថា **tailwind.css** ដែលមានទីតាំងស្ថិតនៅក្នុង Folder **/src/assets/** ដែលមានកូដដូចរូបខាងក្រោម៖



បន្ទាប់មកយើងយក file នោះ ទៅ register នៅក្នុង main.js file ជាការស្រេច។





មេរៀនទី២

CORE CONCEPTS

១. ហ្វូនអក្សរ Typography

ហ្វូនអក្សរមានដូចជា៖ រចនាបថអក្សរ (Font Family), ទំហំអក្សរ (Font size), ពណ៌អក្សរ (Text Color), ទីតាំងអក្សរ (Text Align), កម្រាស់អក្សរ (Font Weight), ចន្លោះបន្ទាត់ (Line Height), គម្លាតអក្សរ (Letter Spacing) ។ល។

- Font Family

`font-sans`

`font-family: ui-sans-serif, system-ui`

`font-serif`

`font-family: ui-serif, Georgia`

`font-mono`

`font-family: ui-monospace`

- Font Size

`text-xs`

`font-size: 0.75rem; line-height: 1rem;`

`text-sm`

`font-size: 0.875rem; line-height: 1.25rem;`

`text-base`

`font-size: 1rem; line-height: 1.5rem;`

`text-lg`

`font-size: 1.125rem; line-height: 1.75rem;`

`text-xl`

`font-size: 1.25rem; line-height: 1.75rem;`

`text-2xl`

`font-size: 1.5rem; line-height: 2rem;`

`text-3x`

`font-size: 1.875rem; line-height: 2.25rem;`

`text-4xl`

`font-size: 2.25rem; line-height: 2.5rem;`

`text-5xl`

`font-size: 3rem; line-height: 1;`

- Text Color

`text-transparent`

`color: transparent;`

`text-white`

`color: rgba(255, 255, 255, 1);`

`text-black`

`color: rgba(0, 0, 0, 1);`

`text-pink-50`

`color: rgba(253, 242, 248, 1);`

`text-pink-100`

`color: rgba(252, 231, 243, 1);`

`text-pink-200`

`color: rgba(251, 207, 232, 1);`

`text-pink-300`

`color: rgba(249, 168, 212, 1);`

`text-pink-400`

`color: rgba(244, 114, 182, 1);`

`text-pink-500`

`color: rgba(236, 72, 153, 1);`

`text-pink-600`

`color: rgba(219, 39, 119, 1);`

`text-pink-700`

`color: rgba(190, 24, 93, 1);`

`text-pink-800`

`color: rgba(157, 23, 77, 1);`

`text-pink-900`

`color: rgba(131, 24, 67, 1);`

នៅពេលដែលយើងចង់ប្រើពណ៌ផ្សេងៗទៀត៖ ពណ៌ក្រហមយើងប្រើ `text-red-[number]`

- Text Align

`text-left`

`text-align: left;`

`text-right`

`text-align: right;`

`text-center`

`text-align: center;`

`text-justify`

`text-align: justify;`

- Font Weight

font-thin	font-weight: 100;
font-extralight	font-weight: 200;
font-light	font-weight: 300;
font-normal	font-weight: 400;
font-medium	font-weight: 500;
font-semibold	font-weight: 600;
font-bold	font-weight: 700;
font-extrabold	font-weight: 800;
font-black	font-weight: 900;
- Line Height

leading-3	line-height: .75rem;
leading-4	line-height: 1rem;
leading-5	line-height: 1.25rem;
leading-6	line-height: 1.5rem;
leading-7	line-height: 1.75rem;
leading-8	line-height: 2rem;
leading-9	line-height: 2.25rem;
leading-10	line-height: 2.5rem;
leading-none	line-height: 1;
leading-tight	line-height: 1.25;
leading-snug	line-height: 1.375;
leading-normal	line-height: 1.5;
leading-relaxed	line-height: 1.625;
leading-loose	line-height: 2;
- Letter Spacing

tracking-tighter	letter-spacing: -0.05em;
tracking-tight	letter-spacing: -0.025em;
tracking-normal	letter-spacing: 0em;
tracking-wide	letter-spacing: 0.025em;
tracking-wider	letter-spacing: 0.05em;
tracking-widest	letter-spacing: 0.1em;

២. ពណ៌ Colors

ពណ៌បែងចែកជា៨ធំៗ និងក្នុង១ពណ៌ៗមានបែងចែកជា១០ទៀត។ ដូចជា៖

Gray colors.coolGray	50	#F9FADF	100	#F3F4F6	200	#E5E7EB	300	#D1D5DB	400	#9CA3AF
	500	#6B7280	600	#4B5563	700	#374151	800	#1F2937	900	#111827
Red colors.red	50	#FEF2F2	100	#FEE2E2	200	#FECACA	300	#FCA5A5	400	#FB8771
	500	#EF4444	600	#DC2626	700	#B91C1C	800	#991B1B	900	#7F1D1D
Yellow colors.amber	50	#FFFBEF	100	#FEF3C7	200	#FDE68A	300	#FCD34D	400	#FBBF24
	500	#F59E0B	600	#D97706	700	#B45309	800	#92400E	900	#78350F
Green colors.emerald	50	#ECFDF5	100	#D1FAE5	200	#A7F3D0	300	#6EE7B7	400	#34D399
	500	#10B981	600	#059669	700	#047857	800	#065F46	900	#064E3B
Blue colors.blue	50	#EFF6FF	100	#DBEAFE	200	#BFD8FE	300	#93C5FD	400	#60A5FA
	500	#3B82F6	600	#2563EB	700	#1D4ED8	800	#1E40AF	900	#1E3A8A
Indigo colors.indigo	50	#EEF2FF	100	#E0E7FF	200	#C7D2FE	300	#A5B4FC	400	#818CF8
	500	#6366F1	600	#4F46E5	700	#4338CA	800	#3730A3	900	#312E81
Purple colors.violet	50	#F5F3FF	100	#EDE9FE	200	#DDD6FE	300	#C4B5FD	400	#A78BFA
	500	#8B5CF6	600	#7C3AED	700	#6D28D9	800	#5B21B6	900	#4C1D95
Pink colors.pink	50	#FDF2F8	100	#FCE7F3	200	#FBC02D	300	#F9A8D4	400	#F472B6
	500	#EC4899	600	#DB2777	700	#BE185D	800	#9D174D	900	#831843

៣. Background

Background គឺជាផ្ទៃខាងក្រោយរបស់ធាតុ (element) ដែលវាមានមុខងារធ្វើឲ្យធាតុនោះមានភាពច្បាស់ឬមិនច្បាស់តាមការលេងពណ៌ឬរូបភាពរបស់យើង។

Background មានដូចជា៖ Background Attachment, Background Color, Background Opacity, Background Gradient ។ល។

- Background Attachment

<code>bg-fixed</code>	<code>background-attachment: fixed;</code>
<code>bg-local</code>	<code>background-attachment: local;</code>
<code>bg-scroll</code>	<code>background-attachment: scroll;</code>

- Background Color

<code>bg-transparent</code>	<code>background-color: transparent;</code>
<code>bg-white</code>	<code>background-color: rgba(255, 255, 255, 1);</code>
<code>bg-black</code>	<code>background-color: rgba(0, 0, 0, 1);</code>
<code>bg-pink-50</code>	<code>background-color: rgba(253, 242, 248, 1);</code>
<code>bg-pink-100</code>	<code>background-color: rgba(252, 231, 243, 1);</code>
<code>bg-pink-200</code>	<code>background-color: rgba(251, 207, 232, 1);</code>
<code>bg-pink-300</code>	<code>background-color: rgba(249, 168, 212, 1);</code>
<code>bg-pink-400</code>	<code>background-color: rgba(244, 114, 182, 1);</code>
<code>bg-pink-500</code>	<code>background-color: rgba(236, 72, 153, 1);</code>
<code>bg-pink-600</code>	<code>background-color: rgba(219, 39, 119, 1);</code>
<code>bg-pink-700</code>	<code>background-color: rgba(190, 24, 93, 1);</code>
<code>bg-pink-800</code>	<code>background-color: rgba(157, 23, 77, 1);</code>
<code>bg-pink-900</code>	<code>background-color: rgba(131, 24, 67, 1);</code>

- Background Opacity

<code>bg-opacity-0</code>	<code>background-color: rgba(---, ---, ---, 0);</code>
<code>bg-opacity-5</code>	<code>background-color: rgba(---, ---, ---, 0.05);</code>
<code>bg-opacity-10</code>	<code>background-color: rgba(---, ---, ---, 0.1);</code>
<code>bg-opacity-20</code>	<code>background-color: rgba(---, ---, ---, 0.2);</code>
<code>bg-opacity-25</code>	<code>background-color: rgba(---, ---, ---, 0.25);</code>
<code>bg-opacity-30</code>	<code>background-color: rgba(---, ---, ---, 0.3);</code>
<code>bg-opacity-40</code>	<code>background-color: rgba(---, ---, ---, 0.4);</code>
<code>bg-opacity-50</code>	<code>background-color: rgba(---, ---, ---, 0.5);</code>
<code>bg-opacity-60</code>	<code>background-color: rgba(---, ---, ---, 0.6);</code>
<code>bg-opacity-70</code>	<code>background-color: rgba(---, ---, ---, 0.7);</code>
<code>bg-opacity-75</code>	<code>background-color: rgba(---, ---, ---, 0.75);</code>
<code>bg-opacity-80</code>	<code>background-color: rgba(---, ---, ---, 0.8);</code>
<code>bg-opacity-90</code>	<code>background-color: rgba(---, ---, ---, 0.9);</code>
<code>bg-opacity-95</code>	<code>background-color: rgba(---, ---, ---, 0.95);</code>
<code>bg-opacity-100</code>	<code>background-color: rgba(---, ---, ---, 1);</code>

- Background Gradient គឺជាល្បាយពណ៌ផ្ទៃខាងក្រោយ។

ទម្រង់នៃការប្រើប្រាស់៖ `bg-gradient-to-[direction] from-[color]-[number] to-[color]-[number]`



- **direction** មានដូចជា t (top), tr (top-right), r (right), br (bottom-right), b (bottom), bl (bottom-left), l (left), tl (top-left) ។
- **color** មានដូចជា black, white, pink, red, yellow, green ។ល។
- **number** មានដូចជា 50, 100, 200, 300, 400, 500, 600, 700, 800, 900។

៤. ទំហំទទឹង Width

Width គឺជារង្វាស់ខ្នាតទៅលើវត្ថុមួយពីម្ខាងទៅម្ខាងតាមរយៈអ័ក្សដេក។

ការប្រើប្រាស់ width ៖

w-0	=>	width: 0px	
w-px	=>	width: 1px	
w-0.5	=>	width: 0.125rem	=> width: 2px
w-1	=>	width: 0.25rem	=> width: 4px
w-4	=>	width: 1rem	=> width: 16px
w-auto	=>	width: auto	
w-1/2	=>	width: 50%	
w-1/3	=>	width: 33.33%	
w-1/4	=>	width: 25%	
w-full	=>	width: 100%	
w-screen	=>	width: 100vw	
w-min	=>	width: min-content	
w-max	=>	width: max-content	

៥. ទំហំកម្ពស់ Height

Height គឺជារង្វាស់ខ្នាតទៅលើវត្ថុមួយពីម្ខាងទៅម្ខាងតាមរយៈអ័ក្សឈរ។

ការប្រើប្រាស់ height ៖

h-0	=>	height: 0px	
h-px	=>	height: 1px	
h-0.5	=>	height: 0.125rem	=> height: 2px
h-1	=>	height: 0.25rem	=> height: 4px
h-4	=>	height: 1rem	=> height: 16px
h-auto	=>	height: auto	
h-1/2	=>	height: 50%	
h-1/3	=>	height: 33.33%	
h-1/4	=>	height: 25%	
h-full	=>	height: 100%	
h-screen	=>	height: 100vh	

៦. Padding

Padding គឺជាគម្លាតរវាង Content និង Border។

នៅក្នុង Tailwindcss Padding បែងចែកជា៖

* ចំណាំ ៖ [px = 1px] [1 = 0.25rem = 4px] [4 = 1rem = 16px]

<code>p-1</code>	=>	padding ឆ្វេង ស្តាំ លើ ក្រោម 4px
<code>px-2</code>	=>	padding ឆ្វេង ស្តាំ 8px
<code>py-3</code>	=>	padding លើ ក្រោម 12px
<code>pr-px</code>	=>	padding ស្តាំ 1px
<code>pl-4</code>	=>	padding ឆ្វេង 16px
<code>pt-5</code>	=>	padding លើ 20px
<code>pb-10</code>	=>	padding ក្រោម 40px

៧. Margin

Margin គឺជាគម្លាតជុំវិញ HTML Element ដែលនៅខាងក្រៅ Border របស់ Content ។ Margin ក៏មិនខុសពី Padding ដែរវាបែងចែកជា៖

<code>m-1</code>	=>	margin ឆ្វេង ស្តាំ លើ ក្រោម 4px
<code>mx-2</code>	=>	margin ឆ្វេង ស្តាំ 8px
<code>my-3</code>	=>	margin លើ ក្រោម 12px
<code>mr-px</code>	=>	margin ស្តាំ 1px
<code>ml-4</code>	=>	margin ឆ្វេង 16px
<code>mt-5</code>	=>	margin លើ 20px
<code>mb-10</code>	=>	margin ក្រោម 40px

៨. Spacing

Space គឺជាគម្លាតជុំវិញ HTML Element ដែលនៅខាងក្រៅ Border របស់ Content ។ និយមន័យរបស់ space ដូចទៅនឹង margin ដែរ ប៉ុន្តែការប្រើប្រាស់របស់វាមានលក្ខណៈខុសគ្នា។ margin គឺប្រើសំរាប់តែ HTML Element តែមួយប៉ុណ្ណោះ ចំណែក space យើងប្រើទៅលើក្រុម HTML Element ។ Space បែងចែកជា២ប្រភេទប៉ុណ្ណោះ៖

<code>space-x-2</code>	=>	គម្លាត ឆ្វេង ស្តាំ 8px
<code>space-y-3</code>	=>	គម្លាត លើ ក្រោម 12px

៩. Border

Border គឺជាបន្ទាត់ជុំវិញធាតុ។ Border មានដូចជា Border Radius, Border Width, Border Color, Border Style ។

- Border Radius

<code>rounded-none</code>	<code>border-radius: 0px;</code>
<code>rounded-sm</code>	<code>border-radius: 0.125rem;</code>



- | | |
|---------------------------|---------------------------------------|
| <code>rounded</code> | <code>border-radius: 0.25rem;</code> |
| <code>rounded-md</code> | <code>border-radius: 0.375rem;</code> |
| <code>rounded-lg</code> | <code>border-radius: 0.5rem;</code> |
| <code>rounded-xl</code> | <code>border-radius: 0.75rem;</code> |
| <code>rounded-2xl</code> | <code>border-radius: 1rem;</code> |
| <code>rounded-3xl</code> | <code>border-radius: 1.5rem;</code> |
| <code>rounded-full</code> | <code>border-radius: 9999px;</code> |
- **Border Width**

<code>border-0</code>	<code>border-width: 0px;</code>
<code>border-2</code>	<code>border-width: 2px;</code>
<code>border-4</code>	<code>border-width: 4px;</code>
<code>border-8</code>	<code>border-width: 8px;</code>
<code>border</code>	<code>border-width: 1px;</code>
 - **Border Color**

<code>border-transparent</code>	<code>border-color: transparent;</code>
<code>border-white</code>	<code>border-color: rgba(255, 255, 255, 1);</code>
<code>border-black</code>	<code>border-color: rgba(0, 0, 0, 1);</code>
<code>border-pink-50</code>	<code>border-color: rgba(253, 242, 248, 1);</code>
<code>border-pink-100</code>	<code>border-color: rgba(252, 231, 243, 1);</code>
<code>border-pink-200</code>	<code>border-color: rgba(251, 207, 232, 1);</code>
<code>border-pink-300</code>	<code>border-color: rgba(249, 168, 212, 1);</code>
<code>border-pink-400</code>	<code>border-color: rgba(244, 114, 182, 1);</code>
<code>border-pink-500</code>	<code>border-color: rgba(236, 72, 153, 1);</code>
<code>border-pink-600</code>	<code>border-color: rgba(219, 39, 119, 1);</code>
<code>border-pink-700</code>	<code>border-color: rgba(190, 24, 93, 1);</code>
<code>border-pink-800</code>	<code>border-color: rgba(157, 23, 77, 1);</code>
<code>border-pink-900</code>	<code>border-color: rgba(131, 24, 67, 1);</code>
 - **Border Style**

<code>border-solid</code>	<code>border-style: solid;</code>
<code>border-dashed</code>	<code>border-style: dashed;</code>
<code>border-dotted</code>	<code>border-style: dotted;</code>
<code>border-double</code>	<code>border-style: double;</code>
<code>border-none</code>	<code>border-style: none;</code>

១០. ក្រឡាចត្រង្គ Grid

Grid គឺជាក្រឡាចត្រង្គ។ នៅលើ Browser គេបែងចែកជាជួរដេកនិងជួរឈរក្នុងមួយជួរដេកមាន៦ក្រឡានិងមួយជួរឈរមាន១២ក្រឡា ។ Grid មានដូចជា៖ Template Columns, Column Span, Template Rows, Row Span, និង Gap ។

- **Template Columns** ក្រឡាចត្រង្គជួរឈរ

<code>grid-cols-1</code>	<code>grid-template-columns: repeat(1, minmax(0, 1fr));</code>
<code>grid-cols-2</code>	<code>grid-template-columns: repeat(2, minmax(0, 1fr));</code>
<code>grid-cols-3</code>	<code>grid-template-columns: repeat(3, minmax(0, 1fr));</code>

<code>grid-cols-4</code>	<code>grid-template-columns: repeat(4, minmax(0, 1fr));</code>
<code>grid-cols-5</code>	<code>grid-template-columns: repeat(5, minmax(0, 1fr));</code>
<code>grid-cols-6</code>	<code>grid-template-columns: repeat(6, minmax(0, 1fr));</code>
<code>grid-cols-7</code>	<code>grid-template-columns: repeat(7, minmax(0, 1fr));</code>
<code>grid-cols-8</code>	<code>grid-template-columns: repeat(8, minmax(0, 1fr));</code>
<code>grid-cols-9</code>	<code>grid-template-columns: repeat(9, minmax(0, 1fr));</code>
<code>grid-cols-10</code>	<code>grid-template-columns: repeat(10, minmax(0, 1fr));</code>
<code>grid-cols-11</code>	<code>grid-template-columns: repeat(11, minmax(0, 1fr));</code>
<code>grid-cols-12</code>	<code>grid-template-columns: repeat(12, minmax(0, 1fr));</code>
<code>grid-cols-none</code>	<code>grid-template-columns: none;</code>

- Column Span

<code>col-auto</code>	<code>grid-column: auto;</code>
<code>col-span-1</code>	<code>grid-column: span 1 / span 1;</code>
<code>col-span-2</code>	<code>grid-column: span 2 / span 2;</code>
<code>col-span-3</code>	<code>grid-column: span 3 / span 3;</code>
<code>col-span-4</code>	<code>grid-column: span 4 / span 4;</code>
<code>col-span-5</code>	<code>grid-column: span 5 / span 5;</code>
<code>col-span-6</code>	<code>grid-column: span 6 / span 6;</code>
<code>col-span-8</code>	<code>grid-column: span 8 / span 8;</code>
<code>col-span-9</code>	<code>grid-column: span 9 / span 9;</code>
<code>col-span-10</code>	<code>grid-column: span 10 / span 10;</code>
<code>col-span-11</code>	<code>grid-column: span 11 / span 11;</code>
<code>col-span-12</code>	<code>grid-column: span 12 / span 12;</code>

- Template Rows ក្រឡាចត្រង់ជួរដេក

<code>grid-rows-1</code>	<code>grid-template-rows: repeat(1, minmax(0, 1fr));</code>
<code>grid-rows-2</code>	<code>grid-template-rows: repeat(2, minmax(0, 1fr));</code>
<code>grid-rows-3</code>	<code>grid-template-rows: repeat(3, minmax(0, 1fr));</code>
<code>grid-rows-4</code>	<code>grid-template-rows: repeat(4, minmax(0, 1fr));</code>
<code>grid-rows-5</code>	<code>grid-template-rows: repeat(5, minmax(0, 1fr));</code>
<code>grid-rows-6</code>	<code>grid-template-rows: repeat(6, minmax(0, 1fr));</code>
<code>grid-rows-none</code>	<code>grid-template-rows: none;</code>

- Row Span

<code>row-auto</code>	<code>grid-row: auto;</code>
<code>row-span-1</code>	<code>grid-row: span 1 / span 1;</code>
<code>row-span-2</code>	<code>grid-row: span 2 / span 2;</code>
<code>row-span-3</code>	<code>grid-row: span 3 / span 3;</code>
<code>row-span-4</code>	<code>grid-row: span 4 / span 4;</code>
<code>row-span-5</code>	<code>grid-row: span 5 / span 5;</code>
<code>row-span-6</code>	<code>grid-row: span 6 / span 6;</code>
<code>row-span-full</code>	<code>grid-row: 1 / -1;</code>

- Gap

<code>gap-0</code>	<code>gap: 0px;</code>
<code>gap-x-0</code>	<code>column-gap: 0px;</code>
<code>gap-y-0</code>	<code>row-gap: 0px;</code>
<code>gap-px</code>	<code>gap: 1px;</code>



<code>gap-x-px</code>	<code>column-gap: 1px;</code>
<code>gap-y-px</code>	<code>row-gap: 1px;</code>

...

<code>gap-0.5</code>	<code>gap: 0.125rem;</code>
<code>gap-1</code>	<code>gap: 0.25rem;</code>
<code>gap-1.5</code>	<code>gap: 0.375rem;</code>
<code>gap-2</code>	<code>gap: 0.5rem;</code>
<code>gap-2.5</code>	<code>gap: 0.625rem;</code>
<code>gap-3</code>	<code>gap: 0.75rem;</code>
<code>gap-3.5</code>	<code>gap: 0.875rem;</code>
<code>gap-4</code>	<code>gap: 1rem;</code>
<code>gap-5</code>	<code>gap: 1.25rem;</code>
<code>gap-6</code>	<code>gap: 1.5rem;</code>
<code>gap-7</code>	<code>gap: 1.75rem;</code>
<code>gap-8</code>	<code>gap: 2rem;</code>
<code>gap-9</code>	<code>gap: 2.25rem;</code>
<code>gap-10</code>	<code>gap: 2.5rem;</code>
<code>gap-11</code>	<code>gap: 2.75rem;</code>
<code>gap-12</code>	<code>gap: 3rem;</code>
<code>gap-14</code>	<code>gap: 3.5rem;</code>
<code>gap-16</code>	<code>gap: 4rem;</code>
<code>gap-20</code>	<code>gap: 5rem;</code>
<code>gap-24</code>	<code>gap: 6rem;</code>
<code>gap-28</code>	<code>gap: 7rem;</code>
<code>gap-32</code>	<code>gap: 8rem;</code>
<code>gap-36</code>	<code>gap: 9rem;</code>
<code>gap-40</code>	<code>gap: 10rem;</code>
<code>gap-44</code>	<code>gap: 11rem;</code>
<code>gap-48</code>	<code>gap: 12rem;</code>
<code>gap-52</code>	<code>gap: 13rem;</code>
<code>gap-56</code>	<code>gap: 14rem;</code>
<code>gap-60</code>	<code>gap: 15rem;</code>
<code>gap-64</code>	<code>gap: 16rem;</code>
<code>gap-72</code>	<code>gap: 18rem;</code>
<code>gap-80</code>	<code>gap: 20rem;</code>
<code>gap-96</code>	<code>gap: 24rem;</code>

១១. Flex Box

Flex Box គឺជាវិធីសាស្ត្របង្កើនមួយវិមាត្រ (one-dimensional layout method) សំរាប់តម្រៀបធាតុជាជួរដេកឬជួរឈរ។ Flex Box មានដូចជា៖ Flex Direction, Flex Wrap, Flex, Flex Grow, Flex Shrink, Justify Content, Align Items ។

- Flex Direction

<code>flex-row</code>	តម្រៀបធាតុតាមជួរដេក
<code>flex-row-reverse</code>	តម្រៀបធាតុតាមជួរដេកបញ្ជាក់
<code>flex-col</code>	តម្រៀបធាតុតាមជួរឈរ
<code>flex-col-reverse</code>	តម្រៀបធាតុតាមជួរឈរបញ្ជាក់
• Flex Wrap	
<code>flex-wrap</code>	ធ្វើឲ្យ child element បន្ទាប់ចុះបន្ទាត់នៅពេល Parent មានទំហំតូចជាង child element
<code>flex-wrap-reverse</code>	ធ្វើឲ្យ child element បន្ទាប់ចុះបន្ទាត់តាមលំដាប់បញ្ជាក់
<code>flex-nowrap</code>	ការពារកុំឲ្យ child element បន្ទាប់ចុះបន្ទាត់នៅពេល Parent element មានទំហំតូចជាង child element
• Flex	
<code>flex-1</code>	ធ្វើឲ្យទំហំរបស់ធាតុកើនឬថយតាមតម្រូវការ
<code>flex-auto</code>	ធ្វើឲ្យទំហំរបស់ធាតុកើនឬថយតាមតម្រូវការហើយបូកបន្ថែមជាមួយទំហំដើមរបស់វា
<code>flex-initial</code>	ធ្វើឲ្យទំហំរបស់ធាតុកើនឬថយតាមតម្រូវការហើយបូកបន្ថែមជាមួយទំហំដើមរបស់វានៅពេលដែលទំហំអេក្រង់កើន
<code>flex-none</code>	ធ្វើឲ្យទំហំរបស់ធាតុមិនកើនមិនថយ
• Flex Grow	
<code>flex-grow-0</code>	រក្សាទំហំរបស់ធាតុឲ្យនូវទំហំដើមនៅពេលអេក្រង់កើន
<code>flex-grow</code>	ធ្វើឲ្យទំហំរបស់ធាតុកើននៅពេលដែលមានចន្លោះទំនេរ
• Flex Shrink	មានលក្ខណៈផ្ទុយពី Flex Grow
<code>flex-shrink-0</code>	រក្សាទំហំរបស់ធាតុឲ្យនូវទំហំដើមនៅពេលអេក្រង់ថយ
<code>flex-shrink</code>	ធ្វើឲ្យទំហំរបស់ធាតុថយតាមតម្រូវការ
• Justify Content	
<code>justify-start</code>	កំណត់ធាតុឲ្យនៅចំនុចចាប់ផ្តើមដំបូងនៅក្នុង Container
<code>justify-end</code>	កំណត់ធាតុឲ្យនៅចំនុចចុងក្រោយនៅក្នុង Container
<code>justify-center</code>	កំណត់ធាតុឲ្យនៅចំនុចកណ្តាលនៅក្នុង Container
<code>justify-between</code>	កំណត់ធាតុឲ្យនៅចន្លោះគ្នានៅក្នុង Container ដោយខាងឆ្វេងធាតុដំបូងនិងខាងស្តាំធាតុចុងក្រោយមិនមានចន្លោះទេ

`justify-around`

កំណត់ធាតុឲ្យនៅចន្លោះគ្នានៅក្នុង Container ដោយខាងឆ្វេងធាតុដំបូងនិងខាងស្តាំធាតុចុងក្រោយមានចន្លោះដូចគ្នា

`justify-evenly`

កំណត់ធាតុឲ្យនៅចន្លោះគ្នាទ្វេដងនៅក្នុង Container ដោយខាងឆ្វេងធាតុដំបូងនិងខាងស្តាំធាតុចុងក្រោយមានចន្លោះដូចគ្នា

- Align Items

`items-start`

កំណត់ធាតុឲ្យនៅចំនុចចាប់ផ្តើមដំបូងនៅក្នុង Container តាមអ័ក្សឈរ

`items-end`

កំណត់ធាតុឲ្យនៅចំនុចចុងក្រោយនៅក្នុង Container តាមអ័ក្សឈរ

`items-center`

កំណត់ធាតុឲ្យនៅចំនុចកណ្តាលនៅក្នុង Container តាមអ័ក្សឈរ

`items-baseline`

កំណត់ធាតុឲ្យនៅចំនុចដើមរបស់វាតាមអ័ក្ស axis នៅក្នុង Container

`items-stretch`

កំណត់ធាតុឲ្យលាតសន្ធឹងតាមអ័ក្ស axis នៅក្នុង Container

១២. Breakpoint

Breakpoint គឺជាចំនុចដែល Web content response យោងទៅតាម width របស់ device នីមួយៗ។ Breakpoint បែងចែកជា៥ទំហំគឺ៖

prefix	Minimum width	CSS
sm	640px	@media (min-width: 640px) { ... }
md	768px	@media (min-width: 768px) { ... }
lg	1024px	@media (min-width: 1024px) { ... }
xl	1280px	@media (min-width: 1280px) { ... }
2xl	1536px	@media (min-width: 1536px) { ... }



ជំពូកទី៣

FIREBASE

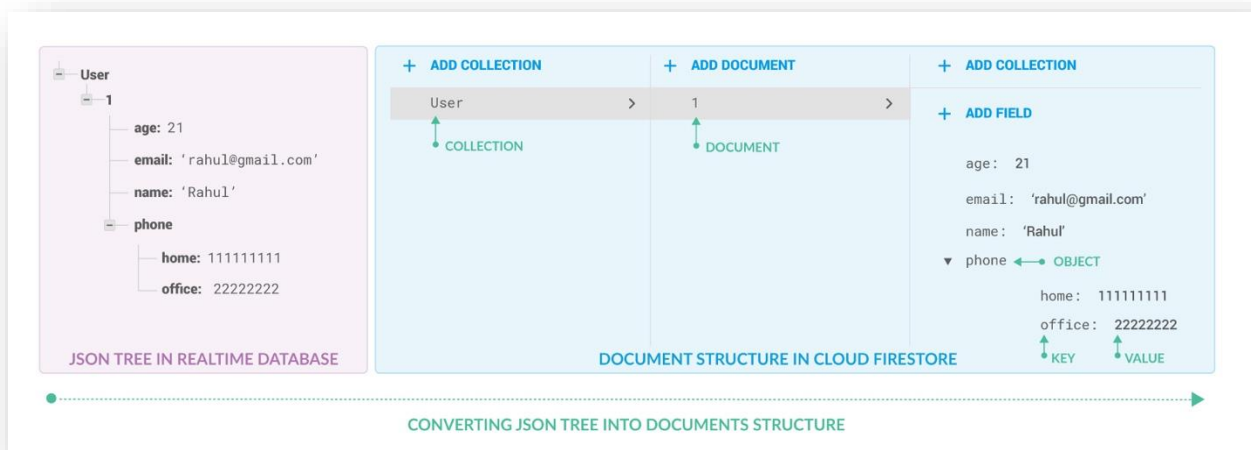
មេរៀនទី១

និយមន័យ និងការដំឡើង

១. និយមន័យ

Firebase គឺជា Backend-as-a-Service (Baas) ។ វាផ្តល់ឲ្យយើងនូវ tools និងសេវាកម្មផ្សេងៗដើម្បីយកមកអភិវឌ្ឍកម្មវិធី (application) ឲ្យមានគុណភាពខ្ពស់និងមានភាពងាយស្រួល ហើយរហ័សទាន់ចិត្ត។ Firebase ត្រូវបានបង្កើតឡើងដោយក្រុមហ៊ុន Google ។ Firebase ដើរតួជា Server Side ផង និងជា Database ផង។ Database នៅក្នុង Firebase បែងចែកជាពីរគឺ Firebase Database និង Cloud Firestore ហើយវាត្រូវបានចាត់ថ្នាក់ជាប្រភេទ NoSQL Database Program ដែលទិន្នន័យរបស់វាត្រូវបានផ្ទុក (store) នៅក្នុង JSON-like Documents។

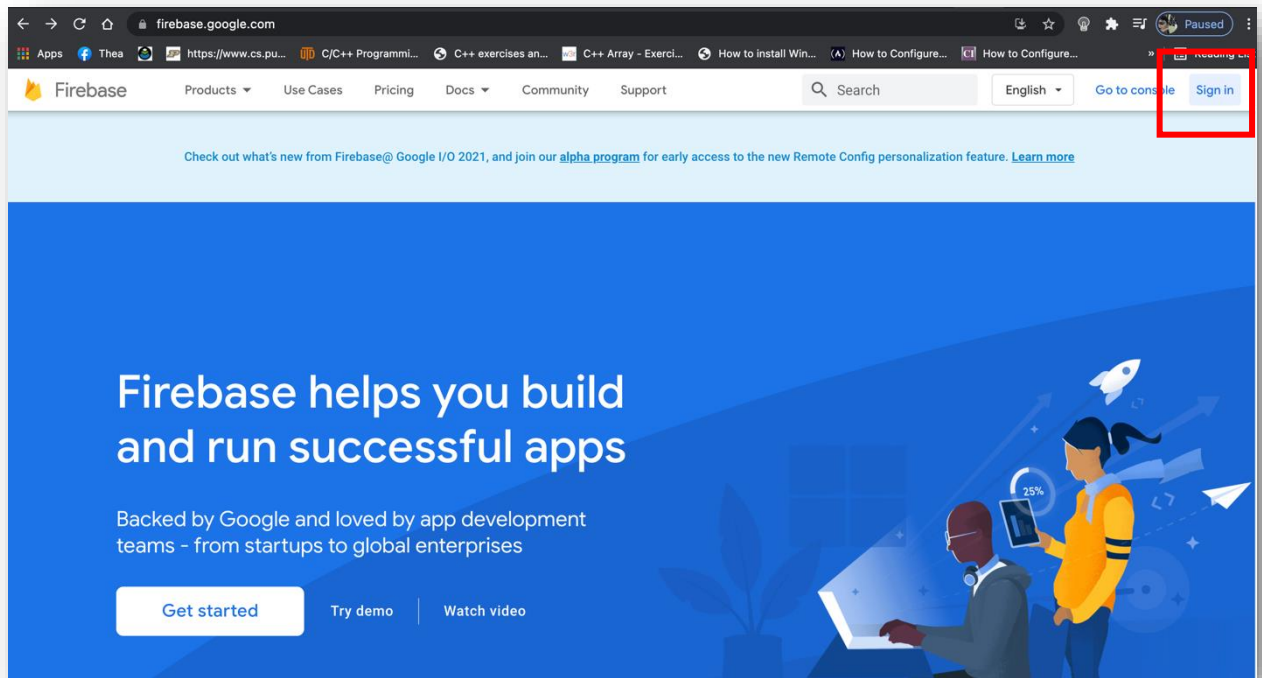
នៅក្នុងមេរៀនរបស់យើងលោកគ្រូនឹងលើកយកតែ Cloud Firestore មកបង្រៀនតែប៉ុណ្ណោះ។ ខាងក្រោមនេះគឺជា Structure របស់ Cloud Firestore ៖



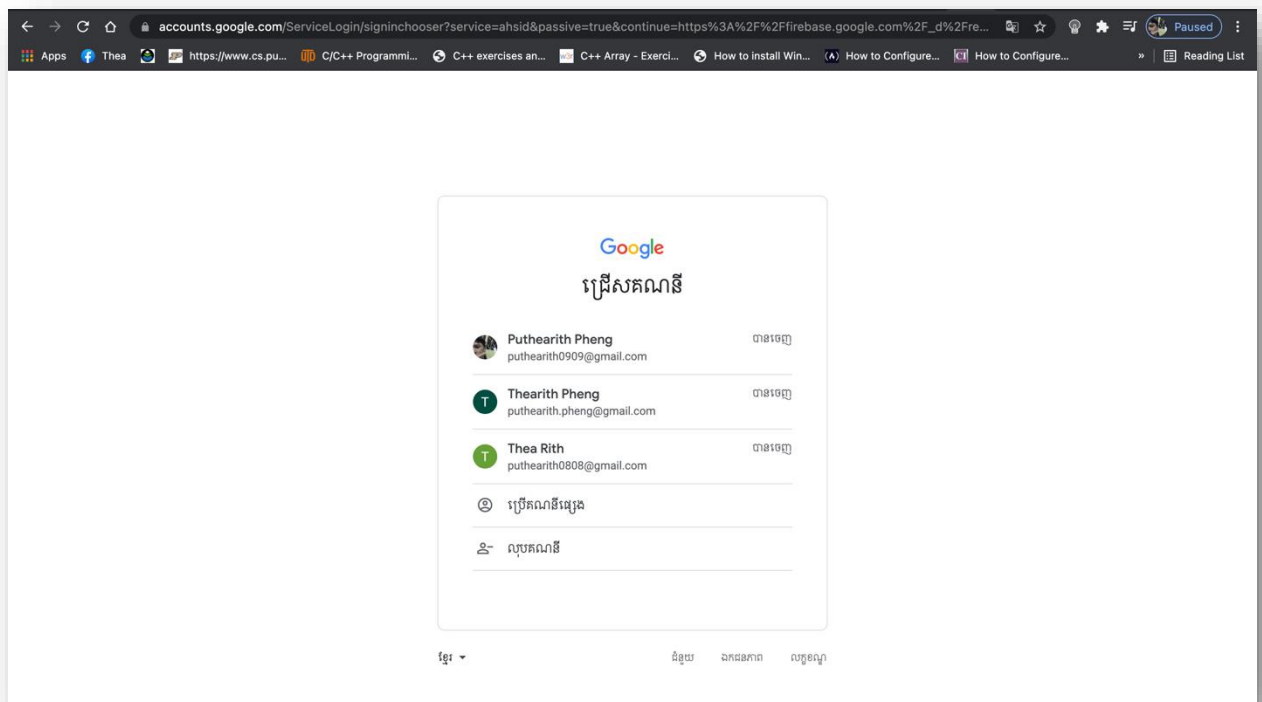
២. ការដំឡើង

មុននឹងដំឡើង Firebase នៅក្នុង Project យើងបានយើងត្រូវធ្វើតាម៣តំណក់ដូចខាងក្រោម៖
តំណក់កាលទី១៖ បង្កើត Account នៅលើ Firebase

1. ចូលទៅកាន់គេហទំព័រ <https://firebase.google.com/>
2. Sign in ជាមួយ Google Account

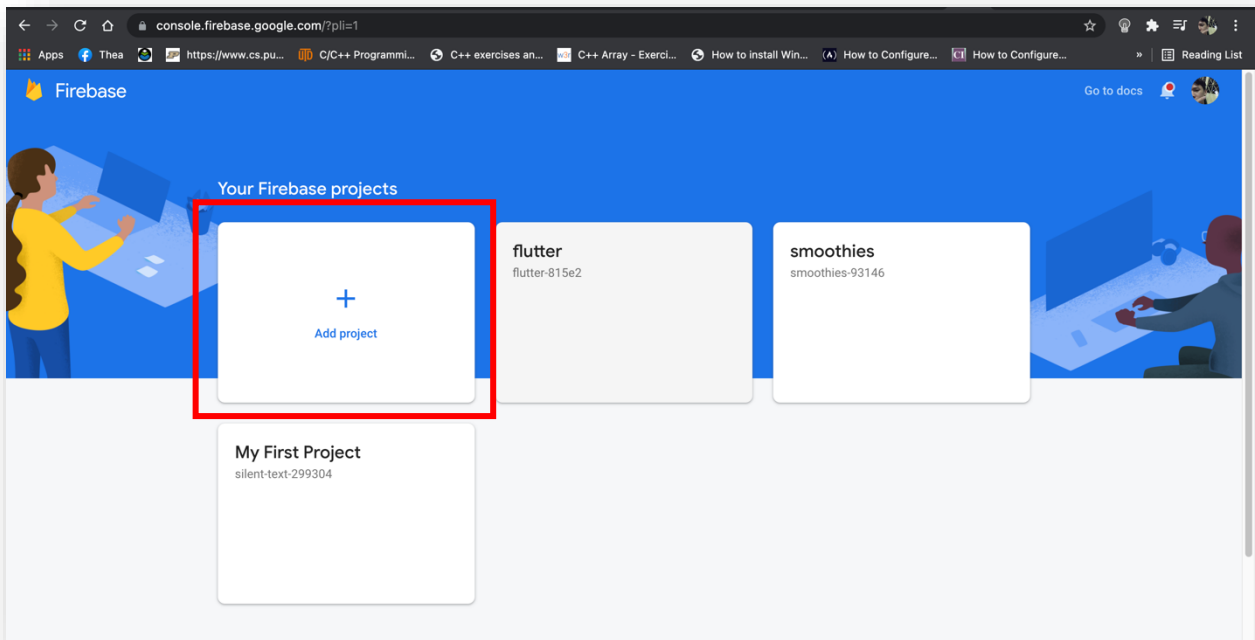


3. ជ្រើសរើសគណនី Gmail របស់អ្នកទាំងអស់គ្នា

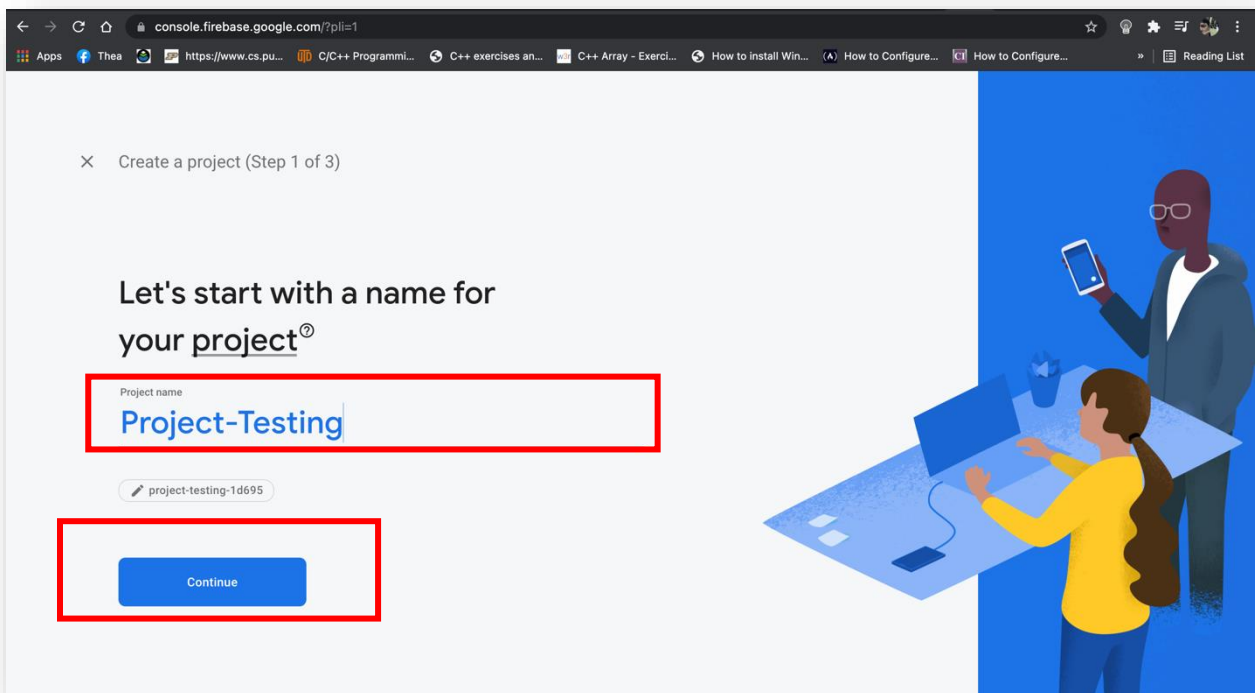


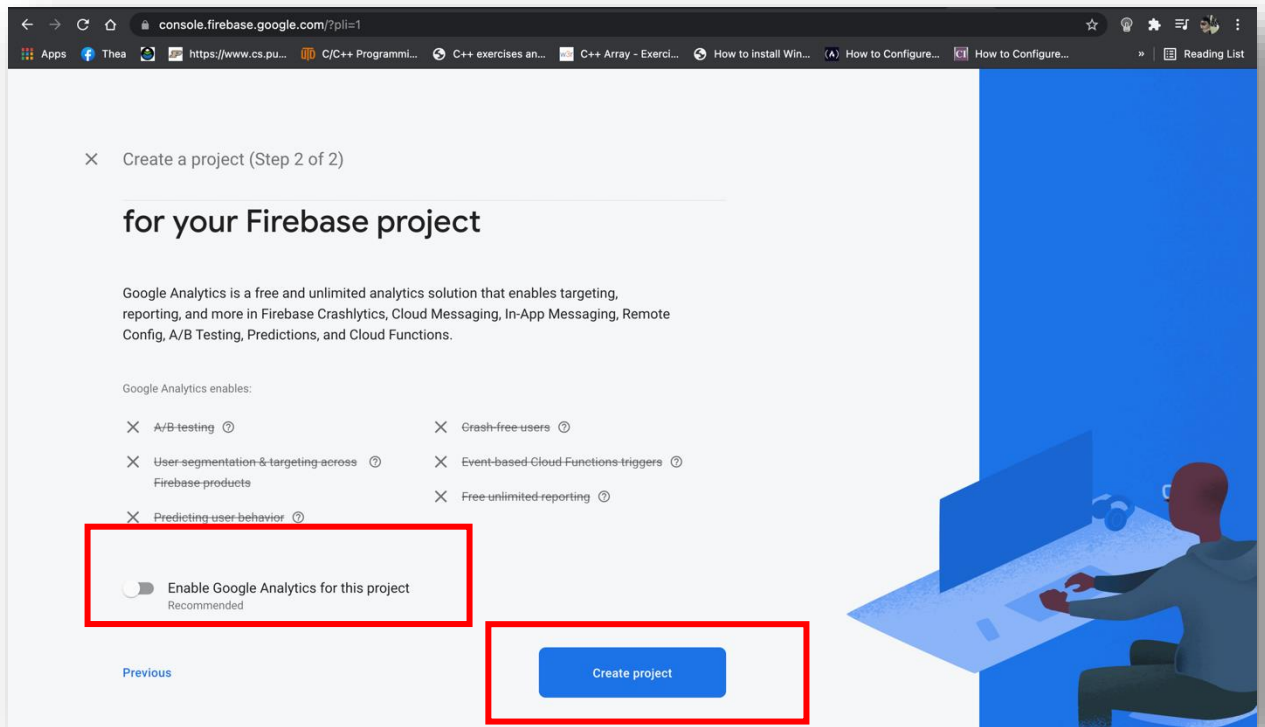
តំណក់កាលទី២៖ បង្កើត Project នៅលើ Firebase

1. ជ្រើសរើសពាក្យថា Add Project



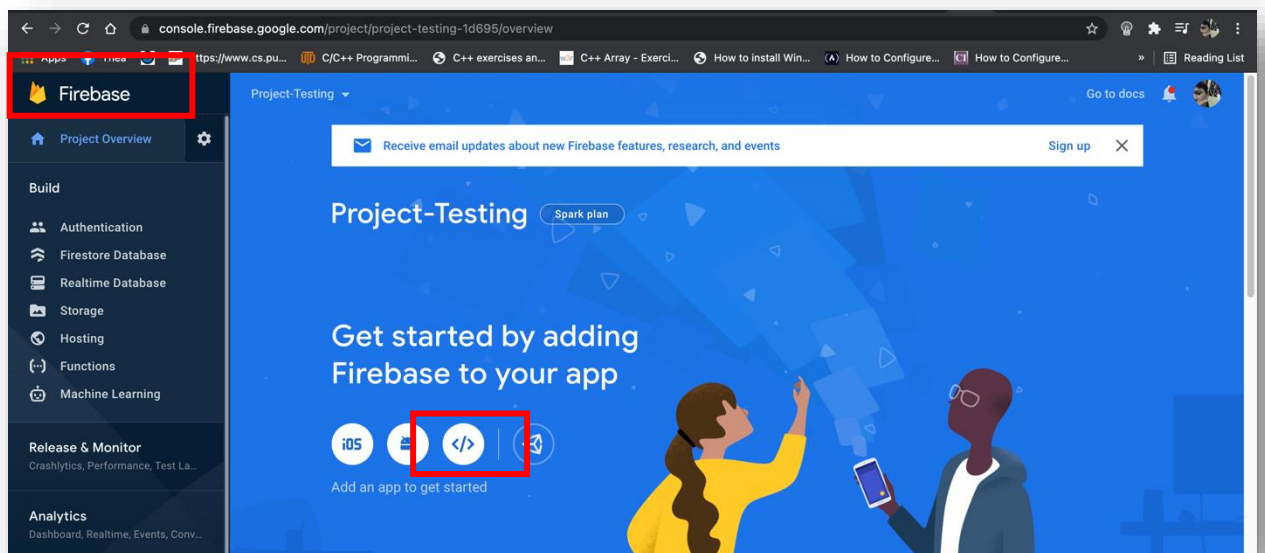
2. ដាក់ឈ្មោះឱ្យ Project





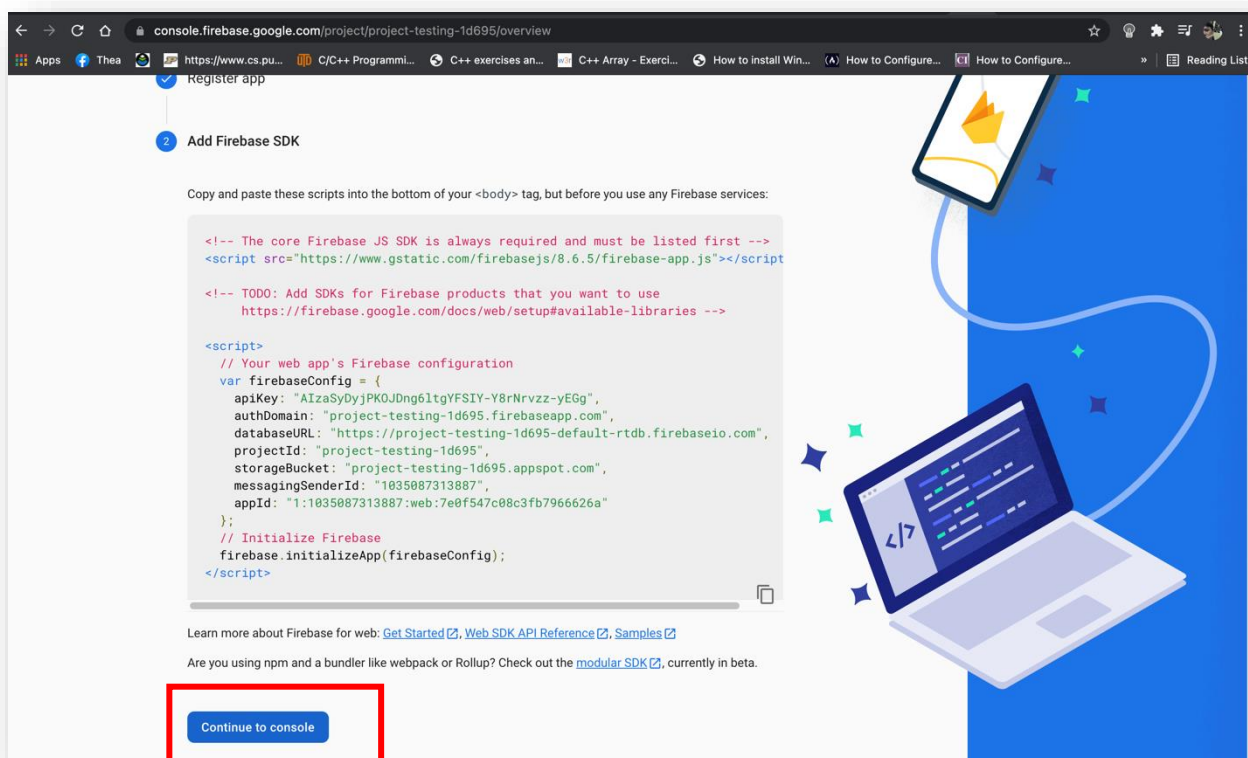
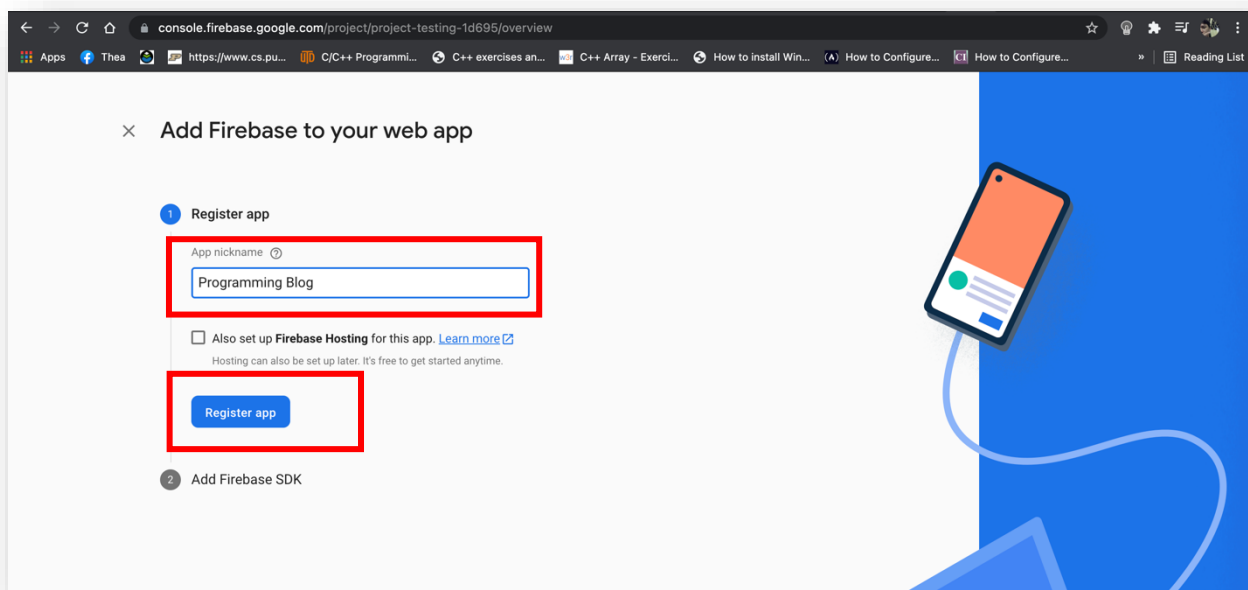
តំណក់កាលទី៣៖ ធ្វើឲ្យ Firebase Project និង Vue Project របស់យើងមានទំនាក់ទំនងនឹងគ្នា។

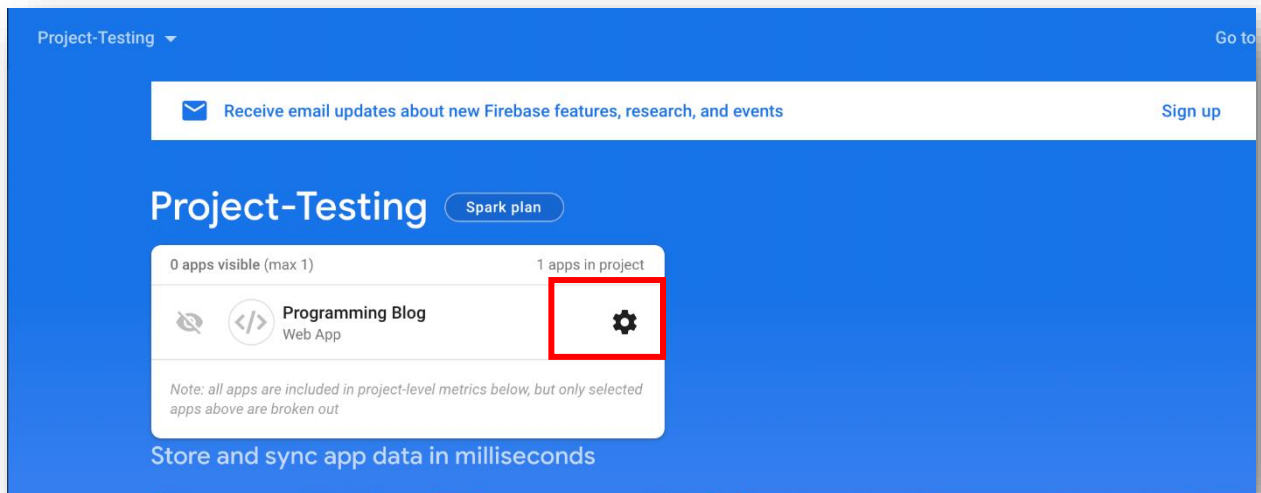
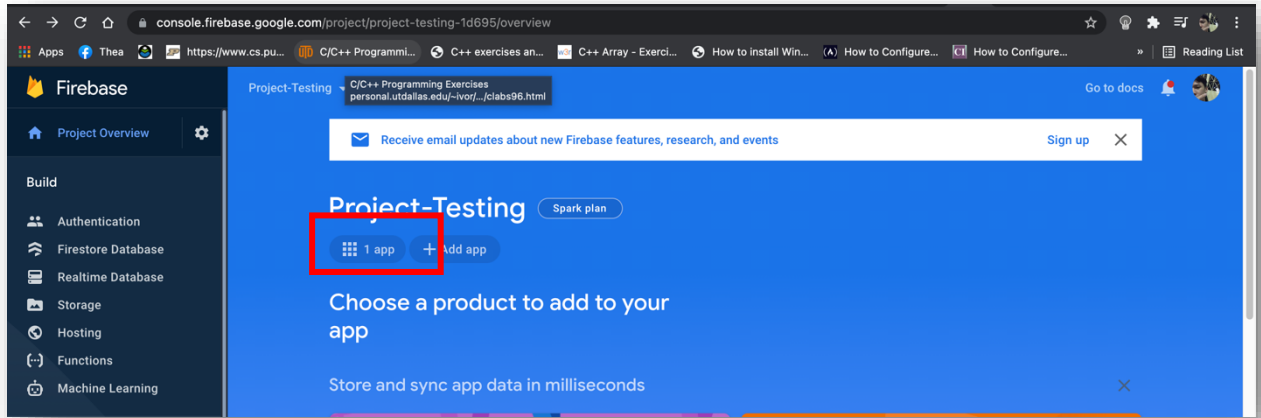
1. បង្កើត Application នៅក្នុង Project Firebase ដែលយើងទើបតែបង្កើត



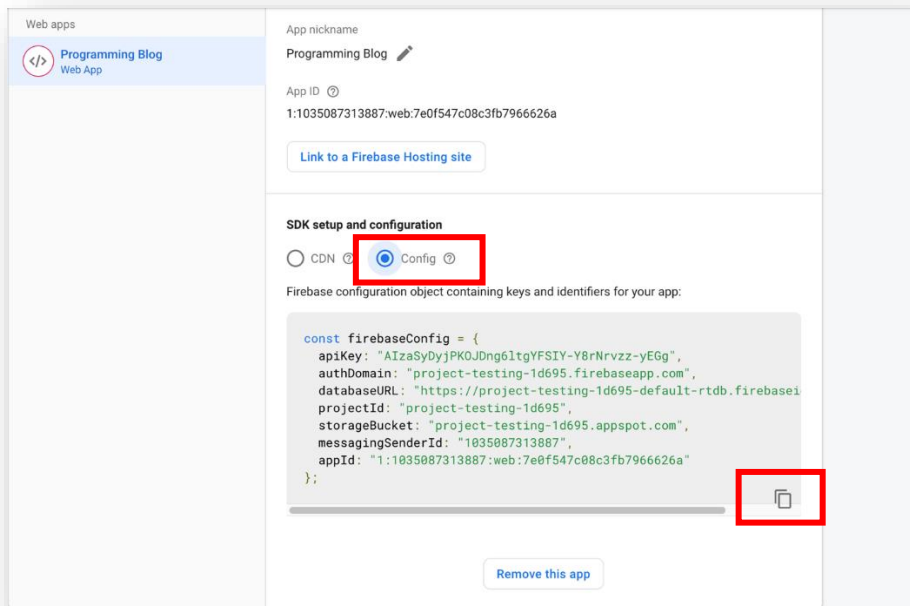


2. ដាក់ឈ្មោះឱ្យ Application របស់យើង





3. ចំណងកូដ firebaseConfig ទៅជាក្នុង Vue Project របស់យើង





4. ចូលទៅកាន់ Terminal ឬ Command Prompt ហើយលើ Vue Project បន្ទាប់ មកវាយបញ្ចូលពាក្យបញ្ជាដើម្បីដំឡើង Firebase Locally នៅលើ Vue Project ។

```

thearith@Phengs-MacBook-Pro ~ /Desktop/eCommerce-project  master ● ?  npm i firebase
added 55 packages, and audited 1560 packages in 1m

6 packages are looking for funding
  run `npm fund` for details

67 vulnerabilities (59 moderate, 8 high)

To address issues that do not require attention, run:
  npm audit fix

To address all issues possible (including breaking changes), run:
  npm audit fix --force

Some issues need review, and may require choosing
a different dependency.

Run `npm audit` for details.
thearith@Phengs-MacBook-Pro ~ /Desktop/eCommerce-project  master ● ?

```

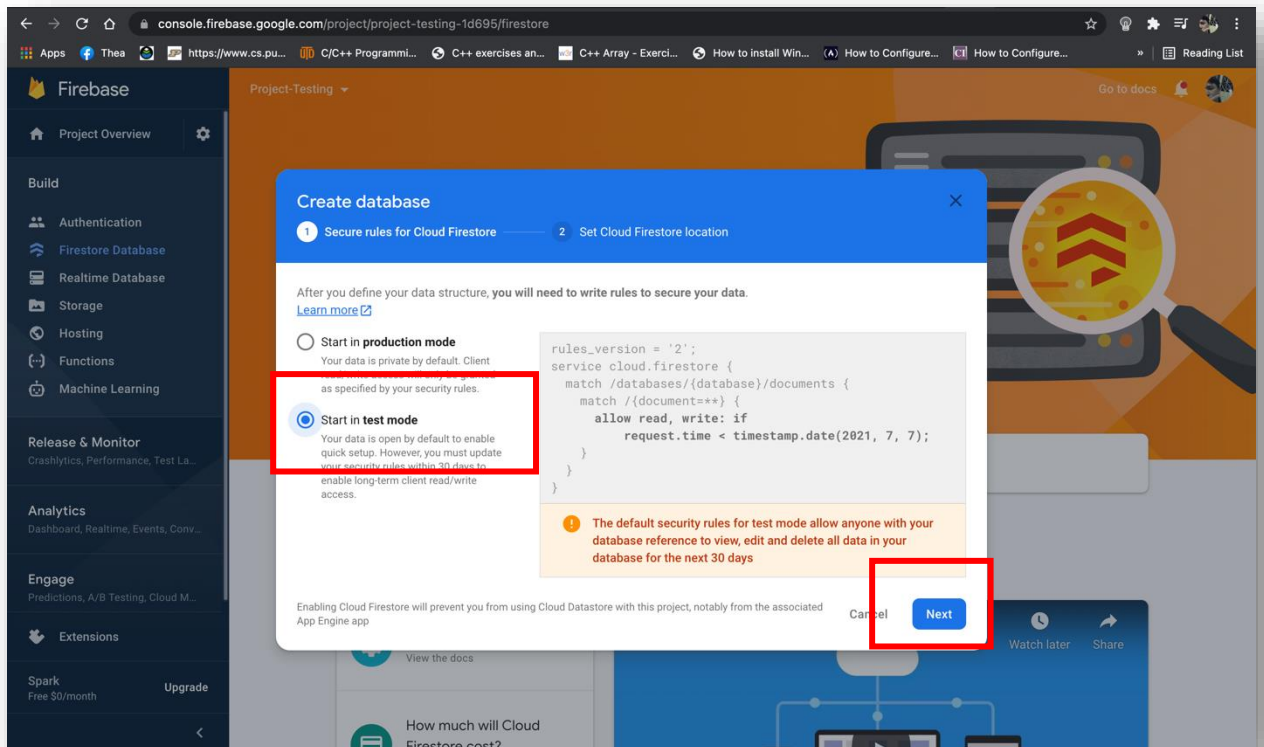
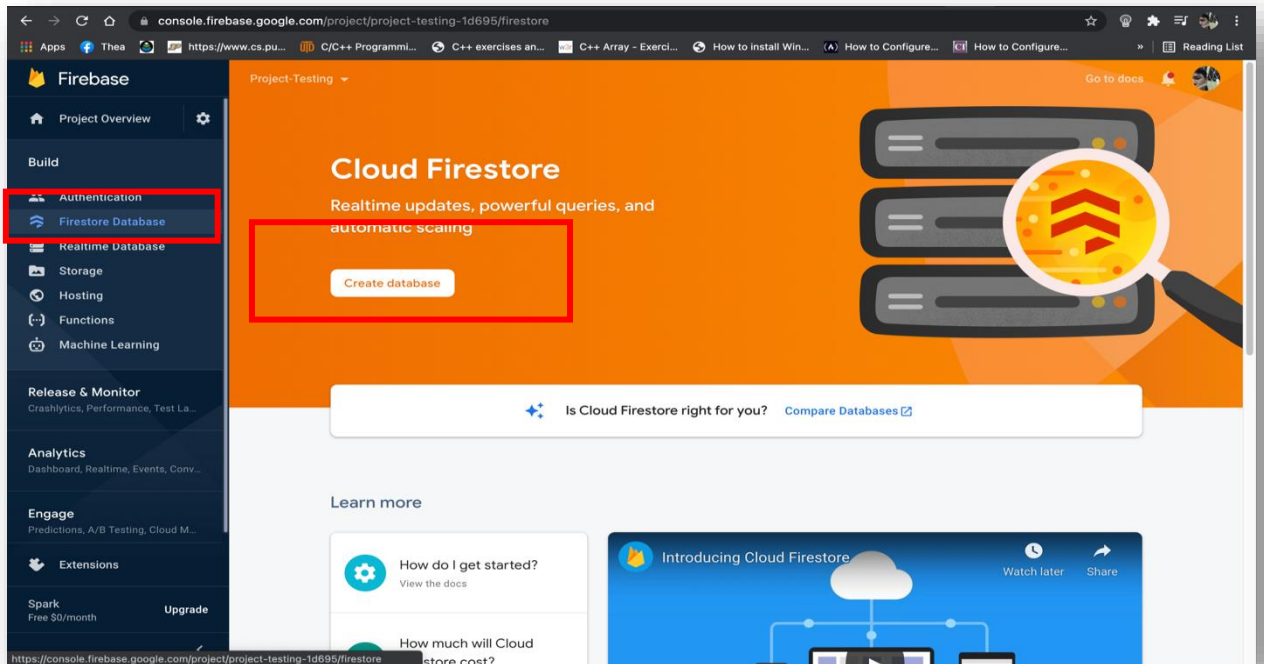
5. នៅក្នុង Vue Project បង្កើត Folder មួយឈ្មោះថា firebase និង File មួយឈ្មោះថា config.js ដែល Folder និង File នោះស្ថិតនៅក្នុង src Folder ។ ក្រោយពីបង្កើតបាន យើងត្រូវសរសេរកូដដូចខាងក្រោម៖

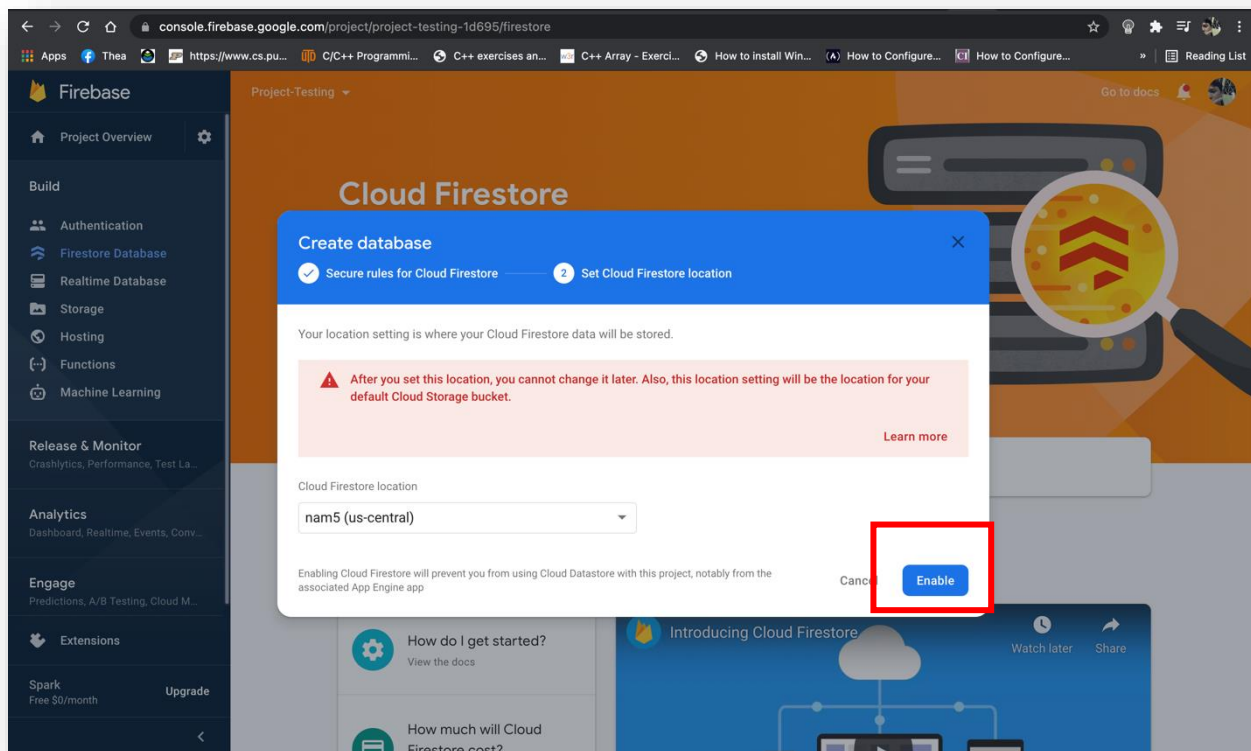
```

0: config.js 0: config.js
1 import firebase from "firebase/app";
2 import "firebase/firestore";
3
4 const firebaseConfig = {
5   apiKey: "AIzaSyDyJPK0JDng6ltgYFSIY-Y8rNrvzz-yEGg",
6   authDomain: "project-testing-1d695.firebaseio.com",
7   databaseURL: "https://project-testing-1d695.firebaseio.com",
8   projectId: "project-testing-1d695",
9   storageBucket: "project-testing-1d695.firebaseio.com",
10  messagingSenderId: "1035087313887",
11  appId: "1:1035087313887:web:7e0f547c08c3fb7966626a"
12 };
13
14 // init firebase
15 firebase.initializeApp(firebaseConfig);
16
17 //init firestore service
18 const projectFirestore = firebase.firestore();
19
20 //export for using later
21 export { projectFirestore };

```


6. បង្កើត Database នៅលើ Firebase Project





មេរៀនទី២

CRUD Operations

នៅក្នុង Computer Programming, Create, Read, Update, and Delete (CRUD) គឺជាមូលដ្ឋានគ្រឹះទាំងបួនសំរាប់ធ្វើការជាមួយ Database ។

១. Create Operation

យើងប្រើប្រាស់ Create Operation ដើម្បីបញ្ជូនទិន្នន័យទៅទុកនៅលើ Cloud Firestore ។

```
import { ref } from "vue";
import { projectFirestore } from "../firebase/config";

const addPost = (collection) => {
  const error = ref(null);

  const load = async (post) => {
    error.value = null;

    try {
      const res = await projectFirestore.collection(collection).add(post);
      return res;
    } catch (err) {
      error.value = err.message;
    }
  };

  return { error, load };
};

export default addPost;
```

២. Read Operation

យើងប្រើប្រាស់ Read Operation ដើម្បីទាញទិន្នន័យពី Cloud Firestore មកបង្ហាញនៅលើ Browser វិញ។

```
import { ref } from "vue";
import { projectFirestore } from "../firebase/config";

const getPosts = (collection) => {
  const error = ref(null);
  const posts = ref(null);

  const gets = async () => {
    error.value = null;

    try {
      const res = await projectFirestore.collection(collection).get();
      posts.value = res.docs.map((doc) => {
```



```

        return { ...doc.data(), id: doc.id };
    });

    } catch (err) {
        error.value = err.message;
        posts.value = null;
    }
};

return { error, posts, gets };
};

export default getPosts;

```

៣. Update Operation

យើងប្រើប្រាស់ Update Operation ដើម្បីកែប្រែទិន្នន័យនៅលើ Cloud Firestore ។

```

import { ref } from "vue";
import { projectFirestore } from "../firebase/config";

const updatePost = (collection) => {
    const error = ref(null);

    const update = async (post) => {
        error.value = null;

        try {
            const res = await projectFirestore
                .collection(collection)
                .doc(post.id).update(post);

            return res;
        } catch (err) {
            error.value = err.message;
        }
    };

    return { error, update };
};

export default updatePost;

```

៤. Delete Operation

យើងប្រើប្រាស់ Delete Operation ដើម្បីលុបទិន្នន័យនៅលើ Cloud Firestore ។

```

import { ref } from "vue";
import { projectFirestore } from "@firebase/config";

const deletePost = (collection) => {
    const error = ref(null);

```

```
const remove = async (id) => {
  error.value = null;

  try {
    const res = await projectFirestore.collection(collection).doc(id).delete();
    return res;
  } catch (err) {
    error.value = err.message;
  }
};

return { remove, error };
};

export default deletePost;
```

ខាងក្រោមគឺជាលទ្ធផលសម្រេចនៃឧទាហរណ៍អំពី CRUD Operations និងតំណភ្ជាប់ដើម្បី
ទៅយក Source Codes ៖

https://github.com/Thearith09/CRUD_Example/

